Leitura — Serviço de CRUD: Ktor × Quarkus (14/09)

Guia de apoio para a aula da Sprint 1. Cobre os conceitos apresentados construindo, do zero, a mesma API de tarefas em dois stacks: Kotlin/Ktor e Java/Quarkus. A ideia é ler antes da aula para chegar com os termos na ponta da língua e voltar a ele durante o projeto, quando surgir uma dúvida.

Exemplos de referência que compilam: `exemplos/ktor-tarefas/` e `exemplos/quarkus-tarefas/` (cada um com `PASSOS.md`). Versões usadas: Kotlin 2.4.10, Ktor 3.5.2, Quarkus 3.28.2 e Java 25. Os comportamentos descritos aqui (códigos de status, mensagens de erro, valores padrão) foram conferidos rodando os exemplos nessas versões; em outras versões, detalhes podem mudar.

Como ler: os capítulos seguem a ordem da aula, mas cada um se sustenta sozinho. Quem já domina HTTP pode começar pelo capítulo 2; quem quer só a parte de Kotlin pode ir direto ao capítulo 4. Os quadros "Erro comum" reúnem o que costuma travar os grupos.

Capítulos:

1. Panorama: HTTP, o exemplo Tarefas, biblioteca × framework
2. Rotas a fundo
3. Configuração: engine, arquivos, perfis e portas
4. Kotlin para quem sabe Java
5. JSON: como o objeto vira resposta (e a requisição vira objeto)
6. Corpo da requisição e status da resposta
7. Arquitetura: regra de dependência, portas e repositório
8. Injeção de dependência: Koin × CDI
9. Concorrência: não travar a thread
10. Rodar, testar e comparar
11. Exercícios e dúvidas frequentes

---

1. Panorama: HTTP, o exemplo Tarefas, biblioteca × framework

A tarefa da aula é uma API HTTP mínima: listar tarefas (`GET /tarefas`) e criar uma (`POST /tarefas`). Os grupos escolhem um stack; vemos os dois lado a lado para separar o que é essencial (todo framework HTTP precisa resolver) do que é estilo de cada um. Antes do código, vale fixar o vocabulário do protocolo, porque os dois frameworks são, no fundo, formas diferentes de ler uma requisição HTTP e escrever uma resposta.

1.1 Uma troca HTTP, por dentro

HTTP é um protocolo de requisição e resposta: o cliente envia uma mensagem, o servidor devolve outra. Em HTTP/1.1 as mensagens são texto. Esta é a criação de uma tarefa:

```
POST /tarefas HTTP/1.1
Host: localhost:8080
Content-Type: application/json
Accept: application/json
Content-Length: 23

{"titulo":"Ler o guia"}
```

A requisição tem quatro partes:

- Linha de requisição: método (`POST`), alvo (`/tarefas`) e versão (`HTTP/1.1`).
- Cabeçalhos: pares `Nome: valor` com metadados. `Content-Type` diz o formato do corpo enviado; `Accept` diz o formato que o cliente aceita receber.
- Uma linha em branco, que separa cabeçalhos do corpo.
- Corpo (opcional): os dados. Um `GET` normalmente não tem corpo.

A resposta segue o mesmo desenho, trocando a primeira linha por uma linha de status:

```
HTTP/1.1 201 Created
Content-Type: application/json
Content-Length: 44

{"id":3,"titulo":"Ler o guia","feita":false}
```

O Ktor responde com `Content-Type: application/json`; o Quarkus acrescenta `; charset=UTF-8`. As duas formas são válidas.

HTTP/2 e HTTP/3 mudam o transporte (as mensagens viajam em quadros binários), mas não o significado: métodos, cabeçalhos e status são os mesmos. Por isso a especificação da semântica (RFC 9110) vale para todas as versões, e o código da aula não muda conforme a versão do protocolo.

Cabeçalhos que aparecem nesta aula:

Cabeçalho	Onde	Para quê
`Host`	requisição	nome e porta do servidor pedido
`Content-Type`	ambos	formato do corpo desta mensagem
`Accept`	requisição	formatos que o cliente aceita na resposta
`Content-Length`	ambos	tamanho do corpo em bytes
`Location`	resposta	URI de um recurso (ex.: o recurso recém-criado)

📖 Ref. MDN — Uma visão geral do HTTP (PT): https://developer.mozilla.org/pt-BR/docs/Web/HTTP/Guides/Overview

📖 Ref. MDN — Mensagens HTTP (PT): https://developer.mozilla.org/pt-BR/docs/Web/HTTP/Guides/Messages

📖 Ref. RFC 9110 — HTTP Semantics: https://www.rfc-editor.org/rfc/rfc9110.html

1.2 Recurso, URI e representação

Três palavras organizam uma API HTTP:

- Recurso é a coisa de que a API trata: uma tarefa, ou a coleção de tarefas.
- URI identifica o recurso: `/tarefas` é a coleção; `/tarefas/3` seria a tarefa 3.
- Representação é a forma como o recurso viaja: aqui, JSON. O mesmo recurso poderia ser representado em XML ou texto; quem escolhe é a negociação de conteúdo (capítulo 5).

A consequência prática é nomear URIs por substantivos (o recurso) e deixar o verbo para o método HTTP. Compare:

```
Bom                         Evitar
GET    /tarefas             GET  /listarTarefas
POST   /tarefas             POST /criarTarefa
GET    /tarefas/3           GET  /tarefa?acao=buscar&id=3
DELETE /tarefas/3           GET  /tarefas/3/apagar
```

Na coluna da direita, a ação está escondida no caminho; ferramentas (caches, proxies, clientes gerados a partir de OpenAPI) deixam de entender o que cada chamada faz.

1.3 Métodos: seguro e idempotente

O método diz a intenção da requisição. Duas propriedades definidas na RFC 9110 importam para projetar a API:

- Seguro: o cliente não pede mudança de estado. O servidor pode registrar log ou contar acessos, mas a requisição não serve para alterar dados.
- Idempotente: repetir a mesma requisição N vezes tem o mesmo efeito pretendido que fazê-la uma vez. A resposta pode mudar (um segundo `DELETE` pode dar `404`), mas o estado final é o mesmo.

Método	Uso típico	Seguro	Idempotente	No exemplo
`GET`	ler um recurso	sim	sim	`GET /tarefas`
`HEAD`	como `GET`, só cabeçalhos	sim	sim	—

Método	Uso típico	Seguro	Idempotente	No exemplo
`POST`	criar ou processar	não	não	`POST /tarefas`
`PUT`	substituir o recurso inteiro	não	sim	exercício
`PATCH`	alterar parte do recurso	não	não	—
`DELETE`	remover	não	sim	exercício
`OPTIONS`	perguntar o que o recurso suporta	sim	sim	—

Por que isso importa: clientes, proxies e bibliotecas repetem requisições quando a rede falha. Repetir um `PUT /tarefas/3` com o mesmo corpo é inofensivo; repetir um `POST /tarefas` cria uma segunda tarefa. Saber disso orienta tanto o desenho das rotas quanto o tratamento de falhas no cliente.

> Erro comum: usar `GET` para mudar estado (`GET /tarefas/3/concluir`). Navegadores fazem pré-carregamento de links e robôs seguem `GET` sem cerimônia; qualquer um deles pode concluir tarefas por acidente. Mudança de estado vai em `POST` , `PUT` , `PATCH` ou `DELETE` .

> Erro comum: usar `POST` para tudo. Funciona, mas descarta a informação que o método carrega (segurança, idempotência, cache) e deixa a API mais difícil de ler.

📖 Ref. MDN — Métodos de requisição HTTP (PT): https://developer.mozilla.org/pt-BR/docs/Web/HTTP/Reference/Methods

📖 Ref. RFC 9110 — Propriedades dos métodos (seguro, idempotente): https://www.rfc-editor.org/rfc/rfc9110.html#section-9.2.1

1.4 Códigos de status

O status resume o resultado em três dígitos. O primeiro dígito dá a classe:

Classe	Significado	Exemplos
`1xx`	informativo, a conversa continua	`100 Continue`
`2xx`	sucesso	`200 OK` , `201 Created` , `204 No Content`
`3xx`	redirecionamento	`301` , `304 Not Modified`
`4xx`	erro do cliente (a requisição tem problema)	`400` , `404` , `415`
`5xx`	erro do servidor (a requisição era válida)	`500` , `503`

A distinção entre `4xx` e `5xx` é a mais útil no dia a dia: `4xx` diz ao cliente "corrija a requisição"; `5xx` diz "o problema é nosso, tente depois". Uma API que devolve `500` para JSON malformado está culpando a si mesma por um erro do cliente.

Códigos que aparecem nesta aula, com as situações conferidas nos dois exemplos:

Status	Quando aparece no exemplo
`200 OK`	`GET /tarefas` bem-sucedido
`201 Created`	`POST /tarefas` criou a tarefa (status definido explicitamente no código)
`204 No Content`	sucesso sem corpo; útil para `DELETE`
`400 Bad Request`	JSON malformado (nos dois); no Ktor, também chave desconhecida ou campo obrigatório ausente (capítulo 5)
`404 Not Found`	caminho sem rota; no Quarkus, também parâmetro de caminho que não converte para o tipo (capítulo 2)
`405 Method Not Allowed`	no Quarkus, caminho existe mas o método não (ex.: `DELETE /tarefas`); o Ktor responde `404` nesse caso
`406 Not Acceptable`	o `Accept` pede um formato que o servidor não produz (ex.: `application/xml`)
`415 Unsupported Media Type`	o `Content-Type` do corpo não é aceito (ex.: `curl -d` sem cabeçalho)
`500 Internal Server Error`	exceção não tratada no código da rota

Sobre o `201`: a RFC 9110 diz que o recurso criado é identificado pelo cabeçalho `Location` ou, na falta dele, pela própria URI da requisição. O exemplo da aula devolve a tarefa criada no corpo, sem `Location`; acrescentar `Location: /tarefas/3` é um refinamento que retomamos no capítulo 6.

📖 Ref. MDN — Códigos de status (PT): https://developer.mozilla.org/pt-BR/docs/Web/HTTP/Reference/Status

📖 Ref. RFC 9110 — 201 Created: https://www.rfc-editor.org/rfc/rfc9110.html#section-15.3.2

1.5 O contrato do exemplo Tarefas

O contrato é o que o cliente pode esperar da API, independente do stack:

Rota	Corpo enviado	Resposta
`GET /tarefas`	—	`200`, lista de tarefas em JSON
`POST /tarefas`	`{"titulo": "..."}`	`201`, a tarefa criada (com `id` e `feita`)

O modelo tem dois tipos. `Tarefa` é o que o servidor guarda e devolve; `NovaTarefa` é o que o cliente envia, sem `id` (quem atribui é o servidor) e sem `feita` (toda tarefa nasce pendente). Separar o tipo de entrada do tipo de saída evita que o cliente escolha o próprio `id`.

```
@Serializable data class Tarefa(val id: Int, val titulo: String, val feita: Boolean = false)
@Serializable data class NovaTarefa(val titulo: String)
```

```
public record Tarefa(int id, String titulo, boolean feita) {}
public record NovaTarefa(String titulo) {}
```

Para testar da linha de comando, com `-i` para ver status e cabeçalhos:

```
curl -i localhost:8080/tarefas
curl -i -X POST localhost:8080/tarefas \
  -H 'Content-Type: application/json' \
  -d '{"titulo":"Ler o guia"}'
```

Troque `8080` por `8081` para o exemplo Quarkus.

> Erro comum: esquecer o `-H 'Content-Type: application/json'`. Com `-d`, o curl envia `Content-Type: application/x-www-form-urlencoded`, e os dois servidores respondem `415 Unsupported Media Type`. O corpo está certo; o rótulo do corpo, não.

📖 Ref. curl — manual (opções `-d`, `-H`, `-i`): https://curl.se/docs/manpage.html

1.6 Biblioteca × framework

A diferença mais citada entre os dois é esta: o Ktor se apresenta como biblioteca, o Quarkus como framework. A distinção clássica é quem chama quem. Você chama uma biblioteca quando precisa dela; um framework chama o seu código nos momentos que ele define. Martin Fowler descreve isso como inversão de controle.

Na prática, os dois invertem algum controle (nenhum servidor HTTP deixa você escrever o laço que aceita conexões), mas em graus bem diferentes:

- No Ktor, você escreve o `main`, escolhe o engine, instala cada plugin com `install(...)` e registra cada rota chamando funções. Se uma linha não foi executada, o recurso não existe. Ler `Aplicacao.kt` de cima para baixo mostra tudo o que a aplicação faz.
- No Quarkus, não há `main` no exemplo. Durante o build, o Quarkus indexa as anotações do projeto, descobre as classes com `@Path`, liga a injeção de dependência e gera o código de

inicialização. Adicionar a extensão `quarkus-rest-jackson` ao `pom.xml` basta para as respostas saírem em JSON. Muito do comportamento vem de convenções e das extensões presentes.

Aspecto	Ktor	Quarkus
Ponto de entrada	`main` escrito por você	gerado pelo Quarkus
Rotas	chamadas de função num DSL	anotações descobertas no build
Ativar JSON	`install(ContentNegotiation) { json() }`	dependência `quarkus-rest-jackson`
Onde erros de montagem aparecem	compilação (tipos do DSL) e execução	compilação e build (ex.: rotas duplicadas falham o build)
Custo de entrada	ler o código basta	conhecer as convenções

Nenhum é melhor em absoluto. O Ktor deixa tudo explícito, ao custo de escrever mais montagem; o Quarkus escreve a montagem por você, ao custo de exigir que você conheça as convenções para saber o que está acontecendo.

📖 Ref. Martin Fowler — Inversion of Control: https://martinfowler.com/bliki/InversionOfControl.html

📖 Ref. Quarkus REST — o que é: https://quarkus.io/guides/rest

1.7 O que todo framework HTTP resolve

Os próximos capítulos seguem este mapa. Em cada linha, o problema é o mesmo; muda a forma:

Problema	Ktor	Quarkus	Capítulo
Mapear método + caminho para código	`routing { get(...) }`	`@Path` + `@GET`	2
Ler parâmetros de caminho e de query	`call.parameters`	`@RestPath`, `@RestQuery`	2
Configurar porta, host, ambiente	código ou `application.conf`	`application.properties` e perfis	3
Converter objeto ↔ JSON	kotlinx.serialization	Jackson	5
Ler o corpo, definir o status	`receive<T>()`, `respond(status, ...)`	parâmetro do método, `Response`	6
Separar rota de acesso a dados	interface de repositório	interface de repositório	7
Montar as dependências	Koin	CDI	8

Problema	Ktor	Quarkus	Capítulo
Atender muitas requisições sem travar	corrotinas	event loop + threads de trabalho	9

---

2. Rotas a fundo

Rota é a regra que liga método + caminho a um trecho de código. Os dois frameworks fazem a mesma coisa com estilos opostos: no Ktor, você constrói uma árvore de rotas chamando funções; no Quarkus, você anota classes e métodos, e o framework monta a árvore.

2.1 Ktor: a árvore de rotas em código

O arquivo `Rotas.kt` do exemplo:

```
fun Application.rotas() {
  val repositorio by inject<RepositorioDeTarefas>()

  routing {
    get("/") { call.respond(mapOf("status" to "no ar")) }

    route("/tarefas") {
      get { call.respond(repositorio.listar()) }          // GET  /tarefas
      post {                                               // POST /tarefas
        val nova = call.receive<NovaTarefa>()
        call.respond(HttpStatusCode.Created, repositorio.adicionar(nova))
      }
    }
  }
}
```

Peça por peça:

- `routing { }` instala o plugin de roteamento (`RoutingRoot`) e abre o bloco onde as rotas são declaradas.
- `route("/tarefas") { }` cria um nó da árvore com o segmento `/tarefas`. Tudo declarado dentro dele herda esse prefixo.
- `get { }` e `post { }` sem caminho respondem no próprio nó. Com caminho, `get("/{id}")` cria um nó filho.

- O bloco passado a `get` / `post` é o handler: o código que roda quando a rota casa. Dentro dele, `call` dá acesso à requisição e à resposta.

A árvore resultante:

```
/                    GET
└── tarefas          GET, POST
```

Os dois estilos abaixo produzem a mesma árvore; o segundo agrupa por recurso e escala melhor quando o número de rotas cresce:

```
// agrupado por método
routing {
  get("/tarefas") { /* ... */ }
  post("/tarefas") { /* ... */ }
}

// agrupado por caminho
routing {
  route("/tarefas") {
    get { /* ... */ }
    post { /* ... */ }
  }
}
```

Quando as rotas crescem, a documentação do Ktor recomenda extrair grupos para funções de extensão de `Route` (o capítulo 4 explica a sintaxe `fun Route.x()`):

```
fun Route.rotasDeTarefas(repositorio: RepositorioDeTarefas) {
  route("/tarefas") {
    get { call.respond(repositorio.listar()) }
    // ...
  }
}

routing {
  rotasDeTarefas(repositorio)
}
```

Como tudo é código, a árvore pode ser montada com `if`, laços ou funções. Isso é poder e risco ao mesmo tempo: uma rota registrada dentro de um `if` que não executou simplesmente não existe.

📖 Ref. Ktor — Routing: https://ktor.io/docs/server-routing.html

2.2 Quarkus: recursos anotados

No Quarkus (Jakarta REST), a unidade é a classe de recurso:

```
@Path("/tarefas")
public class RecursoDeTarefas {

  @Inject RepositorioDeTarefas repositorio;

  @GET
  public List<Tarefa> listar() {                    // GET  /tarefas
    return repositorio.listar();
  }

  @POST
  public Response criar(NovaTarefa nova) {          // POST /tarefas
    Tarefa criada = repositorio.adicionar(nova);
    return Response.status(Response.Status.CREATED).entity(criada).build();
  }
}
```

As regras:

- `@Path` na classe define o prefixo de todos os métodos dela.
- Um método vira endpoint quando tem uma anotação de método HTTP: `@GET`, `@POST`, `@PUT`, `@PATCH`, `@DELETE`, `@HEAD` ou `@OPTIONS`.
- `@Path` no método soma ao da classe: `@Path("/{id}")` num método de uma classe `@Path("/tarefas")` responde em `/tarefas/{id}`.
- A barra inicial é opcional: `@Path("tarefas")` e `@Path("/tarefas")` são equivalentes.
- Um prefixo global (ex.: `/api`) pode ser definido com `@ApplicationPath` numa subclasse de `jakarta.ws.rs.core.Application` ou com a propriedade `quarkus.rest.path`.

Você não registra a classe em lugar nenhum. O Quarkus a encontra no build porque ela tem `@Path`. Um efeito colateral bom dessa análise antecipada: se dois métodos resolvem para o mesmo caminho, método e tipo de conteúdo, o build falha com uma mensagem apontando o conflito, em vez de uma das rotas "sumir" em execução.

📖 Ref. Quarkus REST — mapeamento de URI e métodos HTTP:
https://quarkus.io/guides/rest#declaring-endpoints-uri-mapping

2.3 Parâmetros de caminho

Para buscar uma tarefa por id, o caminho ganha uma variável. O exemplo da aula não tem essa rota; ela é a extensão natural e aparece nos exercícios.

No Ktor, a variável fica entre chaves e é lida com `call.parameters`:

```
route("/tarefas") {
  get("/{id}") {                                    // GET /tarefas/42
    val id = call.parameters["id"]?.toIntOrNull()
    if (id == null) {
      call.respond(HttpStatusCode.BadRequest, "id inválido")
      return@get
    }
    val tarefa = repositorio.listar().find { it.id == id }
    if (tarefa == null) call.respond(HttpStatusCode.NotFound)
    else call.respond(tarefa)
  }
}
```

`call.parameters["id"]` devolve `String?`: o valor vem sempre como texto e pode faltar. A conversão para número é responsabilidade sua; `toIntOrNull()` devolve `null` em vez de lançar exceção. O `return@get` encerra o handler depois de responder (capítulo 4, seção 4.4).

Formas de caminho que o Ktor aceita:

Padrão	Casa com	Não casa com
`/tarefas/{id}`	`/tarefas/42`	`/tarefas`
`/tarefas/{id?}`	`/tarefas/42` e `/tarefas`	— (opcional só no fim do caminho)
`/arquivos/*`	`/arquivos/a` (um segmento qualquer)	`/arquivos`
`/arquivos/{...}`	`/arquivos`, `/arquivos/a/b/c` (resto do caminho)	—
`/arquivos/{partes...}`	como acima, com os segmentos em `getAll("partes")`	—

No Quarkus, a variável vai no `@Path` e chega como parâmetro do método, já convertida:

```java
@GET
@Path("/{id}")
public Response buscar(@RestPath int id) {            // GET /tarefas/42
  return repositorio.listar().stream()
      .filter(t -> t.id() == id)
      .findFirst()
      .map(t -> Response.ok(t).build())
      .orElse(Response.status(Response.Status.NOT_FOUND).build());
}
```

`@RestPath` (pacote `org.jboss.resteasy.reactive`) usa o nome do parâmetro Java para achar a variável do caminho. A anotação padrão do Jakarta REST, `@PathParam("id")`, também funciona, mas exige o nome explícito. O Quarkus aceita até omitir a anotação quando o nome do parâmetro coincide com a variável; deixar `@RestPath` explícito é mais legível. O caminho aceita expressão regular: `@Path("/{id:\\d+}")` só casa com dígitos.

A documentação do Quarkus recomenda compilar com `-parameters` (em Maven, `<maven.compiler.parameters>true</maven.compiler.parameters>`) para que os nomes dos parâmetros estejam disponíveis. No exemplo da aula, `@RestPath int id` funcionou sem essa opção; se um parâmetro sem nome explícito não for reconhecido no seu projeto, comece por aí.

📖 Ref. Ktor — Path parameters: https://ktor.io/docs/server-requests.html

📖 Ref. Quarkus REST — parâmetros da requisição: https://quarkus.io/guides/rest#accessing-request-parameters

2.4 Parâmetros de query

Query é a parte depois do `?`: em `/tarefas?feita=true`, o parâmetro `feita` vale `true`. Serve para filtrar, ordenar e paginar uma coleção, sem criar novas rotas.

```
get {                                                 // GET /tarefas?feita=true
  val feita = call.request.queryParameters["feita"]?.toBooleanStrictOrNull()
  val todas = repositorio.listar()
  call.respond(if (feita == null) todas else todas.filter { it.feita == feita })
}
```

```java
@GET
public List<Tarefa> listar(@RestQuery Boolean feita) {  // GET /tarefas?feita=true
  var todas = repositorio.listar();
  return feita == null ? todas : todas.stream().filter(t -> t.feita() == feita).toList();
}
```

No Quarkus, o tipo `Boolean` (e não `boolean`) permite distinguir "não enviado" (`null`) de `false`. No Ktor, `call.parameters` também inclui os parâmetros de query; usar `call.request.queryParameters` deixa claro de onde o valor vem.

Uma decisão de projeto aparece aqui: o que fazer com `?feita=talvez`? O código Ktor acima ignora o filtro (o valor inválido vira `null`). Responder `400` com uma mensagem clara é mais honesto com o cliente. Escolha uma política e aplique em todas as rotas.

📖 Ref. Ktor — Query parameters: https://ktor.io/docs/server-requests.html

2.5 Tipos de conteúdo: `@Produces` e `@Consumes`

Uma rota também pode depender do formato: `@Produces` declara o que o método gera (casa com o `Accept` do cliente) e `@Consumes` declara o que ele aceita no corpo (casa com o `Content-Type`).

O exemplo não usa nenhuma das duas porque os padrões do Quarkus resolvem:

- Método que devolve `String`: `text/plain`.
- Método que devolve uma classe da aplicação, `List`, `Set`, `Map` ou `Collection`, com `quarkus-rest-jackson` presente: `application/json`.

Declarar explicitamente vale a pena quando você quer restringir. Um caso conferido no exemplo: sem `@Consumes`, um `POST` com `Content-Type: text/plain` e corpo `{"titulo":"Ler"}` foi aceito com `201`, e o texto inteiro virou o título. Com `@Consumes(MediaType.APPLICATION_JSON)`, a mesma requisição recebe `415`.

```java
@POST
@Consumes(MediaType.APPLICATION_JSON)
@Produces(MediaType.APPLICATION_JSON)
public Response criar(NovaTarefa nova) { /* ... */ }
```

Anotações na classe valem para todos os métodos; no método, sobrescrevem as da classe.

No Ktor, quem decide o formato é o plugin `ContentNegotiation`, a partir de `Accept` e `Content-Type` (capítulo 5). Para restringir uma rota a um formato, há seletores no DSL:

```
route("/tarefas") {
  accept(ContentType.Application.Json) {
    get { call.respond(repositorio.listar()) }
  }
}
```

Com esse seletor, `Accept: text/plain` recebe `406`; sem cabeçalho `Accept`, a rota responde normalmente.

📖 Ref. Quarkus REST — content types: https://quarkus.io/guides/rest#declaring-endpoints-representation-content-types

📖 Ref. MDN — Negociação de conteúdo (PT): https://developer.mozilla.org/pt-BR/docs/Web/HTTP/Guides/Content_negotiation

2.6 O mesmo pedido, respostas diferentes

Os dois frameworks concordam no caminho feliz e divergem nas bordas. Situações conferidas com os exemplos:

Requisição	Ktor	Quarkus
`GET /tarefas/` (barra final)	`404`: caminhos com e sem barra são diferentes	`200`: mesmo recurso
`DELETE /tarefas` (caminho existe, método não)	`404`	`405 Method Not Allowed`
`GET /tarefas/abc` com id inteiro	depende do seu código: `400` no exemplo da 2.3	`404`: a conversão falha e a rota não casa
`GET /tarefas` com `Accept: application/xml`	`406`	`406`

No Ktor, o plugin `IgnoreTrailingSlash` faz caminhos com e sem barra final serem tratados como iguais. O `404` do Quarkus para `abc` segue a especificação Jakarta REST; se o contrato da API pede `400` nesse caso, use `String` no parâmetro e converta você mesmo, ou trate a exceção (capítulo 6).

A lição para o projeto: o contrato precisa dizer o que acontece nas bordas, e um teste por borda evita surpresa ao trocar de stack ou de versão.

2.7 Erros comuns com rotas

> Erro comum (Ktor): converter com `!!` e `toInt()`. `call.parameters["id"]!!.toInt()` com `/tarefas/abc` lança `NumberFormatException`, e o cliente recebe `500`. O erro é do cliente, mas o servidor assume a culpa. Use `toIntOrNull()` e responda `400`.

> Erro comum (Ktor): responder e esquecer de sair. `call.respond(...)` não encerra o handler. Sem `return@get`, o código continua: o cliente recebe a primeira resposta, mas o que vem depois (gravar, alterar, logar) ainda executa.

> Erro comum (Ktor): registrar a rota fora do lugar certo. Uma função `fun Route.rotasDeTarefas()` que nunca é chamada dentro de `routing { }` não dá erro de compilação; a rota só não existe, e toda requisição recebe `404`. Ao ver `404` inesperado, confira se a função foi chamada.

> Erro comum (Quarkus): nome do parâmetro diferente da variável. Em `@Path("/{id}")` com `buscar(int codigo)` sem anotação, `codigo` não é reconhecido como parâmetro de caminho: parâmetro sem anotação é tratado como corpo da requisição. Use `@RestPath` com o mesmo nome, ou `@PathParam("id") int codigo`.

> Erro comum (os dois): testar só pelo navegador. O navegador faz `GET` e manda `Accept` próprio. Para `POST`, cabeçalhos e status, use `curl -i` ou o cliente HTTP da IDE.

---

3. Configuração: engine, arquivos, perfis e portas

Toda aplicação precisa responder a perguntas que não são regra de negócio: em que porta escutar? em que endereço? qual servidor HTTP usar? como mudar isso entre a máquina do aluno, os testes e a produção, sem recompilar? Os dois stacks respondem de formas diferentes.

3.1 Engines do Ktor

O Ktor separa a aplicação (rotas e plugins) do engine, o componente que abre a porta, aceita conexões e interpreta o protocolo. A mesma aplicação roda sobre engines diferentes, trocando uma dependência e uma linha:

Engine	Dependência	Plataformas	HTTP/2
Netty	`ktor-server-netty`	JVM	sim
Jetty	`ktor-server-jetty-jakarta`	JVM	sim
Tomcat	`ktor-server-tomcat-jakarta`	JVM	sim
CIO	`ktor-server-cio`	JVM, Native, GraalVM, JS, WasmJs	não

O exemplo usa CIO (Coroutine-based I/O), o mesmo do MUSI. Motivos:

- É escrito em Kotlin sobre corrotinas, o mesmo modelo de concorrência dos handlers (capítulo 9). Não há uma segunda biblioteca de rede com modelo próprio por baixo.
- Tem menos dependências transitivas que Netty, Jetty ou Tomcat.
- Roda também fora da JVM (Kotlin/Native, GraalVM), o que mantém portas abertas.

A limitação é a falta de HTTP/2. Numa API atrás de um proxy reverso (que costuma falar HTTP/2 com o navegador e HTTP/1.1 com a aplicação) ou em desenvolvimento local, isso não pesa. A documentação e o gerador de projetos do Ktor usam Netty nos exemplos; trocar é mudar a dependência e o primeiro argumento de `embeddedServer`:

```
embeddedServer(CIO, port = 8080) { modulo() }     // io.ktor.server.cio.CIO
embeddedServer(Netty, port = 8080) { modulo() }   // io.ktor.server.netty.Netty
```

O Quarkus não expõe essa escolha: o servidor HTTP é sempre o do Vert.x (sobre Netty), configurado pelas propriedades `quarkus.http.*`.

📖 Ref. Ktor — Server engines: https://ktor.io/docs/server-engines.html

3.2 Ktor: `embeddedServer` (código) × `EngineMain` (arquivo)

Há dois jeitos de subir um servidor Ktor.

Com `embeddedServer`, a configuração é código. É o que o exemplo faz:

```
fun main() {
  embeddedServer(CIO, port = 8080, host = "0.0.0.0") { modulo() }
    .start(wait = true)
}
```

`start(wait = true)` bloqueia a thread principal até o servidor parar; sem o `wait`, o `main` terminaria e o processo sairia. Se `port` for omitido, o padrão é `80`, que costuma estar ocupado ou exigir privilégio de administrador; por isso a porta aparece sempre explícita.

Com `EngineMain`, a configuração vem de um arquivo `application.conf` (HOCON) ou `application.yaml` em `src/main/resources`. O `main` só delega:

```
fun main(args: Array<String>): Unit = io.ktor.server.cio.EngineMain.main(args)
```

```
ktor {
  deployment {
    port = 8080
    port = ${?PORT}
  }
  application {
    modules = [ br.ufrn.exemplo.tarefas.AplicacaoKt.modulo ]
  }
}

tarefas {
  saudacao = "Olá do application.conf"
}
```

Três detalhes do arquivo:

- `port = ${?PORT}` usa a variável de ambiente `PORT` se ela existir; o `?` torna a substituição opcional, e a linha anterior (`8080`) fica como padrão. Rodar com `PORT=8095` sobe o servidor na 8095 (conferido).
- `modules` lista as funções `Application.xxx()` a executar, pelo nome completo. O `AplicacaoKt` vem de uma regra do Kotlin: funções de nível superior do arquivo `Aplicacao.kt` são compiladas numa classe chamada `AplicacaoKt` (seção 4.3). É o mesmo nome que aparece no `mainClass` do `build.gradle.kts`.
- `tarefas.saudacao` é uma propriedade da própria aplicação, lida no código com `environment.config.propertyOrNull("tarefas.saudacao")?.getString()`.

Com `EngineMain`, o arquivo empacotado pode ser sobrescrito na linha de comando (`-port=9090`, `-config=outro.conf`, `-P:ktor.deployment.port=9090`), sem recompilar.

	embeddedServer	**EngineMain**
Onde fica a configuração	argumentos em código	`application.conf` / `.yaml`
Mudar a porta sem recompilar	só se o código ler variável de ambiente	`${?PORT}` ou `-port=`
Módulos	chamados no bloco	listados em `ktor.application.modules`
Quando preferir	exemplos, testes, apps pequenas	apps com vários ambientes

No `embeddedServer`, dá para ler a porta do ambiente com Kotlin puro:

```
val porta = System.getenv("PORT")?.toIntOrNull() ?: 8080
embeddedServer(CIO, port = porta, host = "0.0.0.0") { modulo() }.start(wait = true)
```

> Erro comum: criar `application.conf` num projeto que sobe com `embeddedServer`. O arquivo não é lido. No teste feito com o exemplo, `environment.config.propertyOrNull("tarefas.saudacao")` devolveu `null`. Ou você passa a usar `EngineMain`, ou lê a configuração de outro jeito (variável de ambiente, por exemplo).

> Erro comum: guardar segredo no arquivo. Senha de banco e chave de API não vão em `application.conf` versionado. Use `${?DB_PASSWORD}` e defina a variável no ambiente.

📖 Ref. Ktor — Configuração em código: https://ktor.io/docs/server-configuration-code.html

📖 Ref. Ktor — Configuração em arquivo: https://ktor.io/docs/server-configuration-file.html

📖 Ref. Ktor — Módulos: https://ktor.io/docs/server-modules.html

3.3 Quarkus: `application.properties`

No Quarkus, a configuração fica em `src/main/resources/application.properties`. O exemplo tem uma linha:

```
# Porta 8081 para rodar lado a lado com o exemplo Ktor (8080).
quarkus.http.port=8081
```

As propriedades do próprio Quarkus começam com `quarkus.`; as da aplicação podem ter qualquer prefixo. Para ler uma propriedade própria num bean:

```
tarefas.saudacao=Olá do application.properties
```

```
@ConfigProperty(name = "tarefas.saudacao")   // org.eclipse.microprofile.config.inject
String saudacao;
```

A mesma propriedade pode vir de várias fontes. O Quarkus consulta, da maior para a menor prioridade:

Prioridade	Fonte	Exemplo
400	propriedade de sistema	`java -Dquarkus.http.port=9090 -jar ...`
300	variável de ambiente	`QUARKUS_HTTP_PORT=9090`
295	arquivo `.env` no diretório atual	`QUARKUS_HTTP_PORT=9090`
260	`config/application.properties` no diretório atual	`quarkus.http.port=9090`
250	`application.properties` no classpath	o arquivo do projeto

Ou seja: o arquivo do projeto é o padrão, e o ambiente sobrescreve. O nome da variável de ambiente sai do nome da propriedade trocando tudo que não é letra ou dígito por `_` e passando para maiúsculas: `quarkus.http.port` vira `QUARKUS_HTTP_PORT`. No teste com o jar empacotado, `QUARKUS_HTTP_PORT=8096` subiu o servidor na 8096 mesmo com `8081` no arquivo.

📖 Ref. Quarkus — Configuration reference (fontes e variáveis de ambiente): https://quarkus.io/guides/config-reference#configuration-sources

3.4 Perfis do Quarkus

Perfil é um conjunto de valores ativado conforme o modo de execução. O Quarkus tem três perfis padrão:

- `dev`: ativo em `mvn quarkus:dev`.
- `test`: ativo ao rodar os testes.
- `prod`: ativo em qualquer outra execução (o jar ou o executável nativo).

Um valor específico de perfil leva o prefixo `%perfil.`:

```
tarefas.saudacao=Olá do application.properties
%test.tarefas.saudacao=Olá do perfil de teste
%dev.quarkus.log.level=DEBUG
```

Nos testes, `saudacao` vale `Olá do perfil de teste`; em produção, o valor sem prefixo (conferido). Se um perfil não define a propriedade, vale o valor sem prefixo. Também é possível separar em arquivos `application-{perfil}.properties` e criar perfis próprios (ex.: `staging`), ativados com `quarkus.profile=staging`.

O Ktor não tem perfis embutidos. O equivalente é uma propriedade própria alimentada por variável de ambiente (`ktor { environment = ${?KTOR_ENV} }`) e um `when` no código, ou arquivos separados passados com `-config=`.

📖 Ref. Quarkus — Profiles: https://quarkus.io/guides/config-reference#profiles

3.5 Host: `localhost` × `0.0.0.0`

O host diz em qual interface de rede o servidor escuta:

- `localhost` (ou `127.0.0.1`): só a própria máquina alcança o servidor.
- `0.0.0.0`: todas as interfaces; outras máquinas da rede (e o mundo de fora de um contêiner) alcançam.

Os padrões diferem:

	Ktor (`embeddedServer`)	**Quarkus**
Padrão	`0.0.0.0`	`localhost` em dev e test; `0.0.0.0` em prod
No exemplo	`host = "0.0.0.0"` explícito	padrão (o log em dev mostra `http://localhost:8081`)

O padrão do Quarkus em dev evita que colegas na mesma rede acessem sua máquina de desenvolvimento; em prod, `0.0.0.0` facilita rodar em contêiner.

> Erro comum: rodar a aplicação num contêiner escutando em `localhost`. Dentro do contêiner, `localhost` é o próprio contêiner. A porta mapeada no host não chega ao processo e a conexão é recusada. Dentro de contêiner, escute em `0.0.0.0`.

📖 Ref. Quarkus — HTTP reference: https://quarkus.io/guides/http-reference

3.6 Portas e a porta de teste do Quarkus

O Quarkus usa `8080` por padrão e roda os testes numa porta separada, `8081` (`quarkus.http.test-port`), para não colidir com a aplicação em execução. O exemplo da aula escolheu `8081` para a aplicação, para rodar ao lado do Ktor. As duas escolhas se chocam:

```
mvn quarkus:dev   → aplicação em :8081
mvn test          → testes tentam subir em :8081
                  → QuarkusBindException: Port already bound: 8081
```

O conflito foi reproduzido com o exemplo. A saída é mover a porta de teste:

```
quarkus.http.port=8081
quarkus.http.test-port=8083     # ou 0 para uma porta livre aleatória
```

No Ktor, os testes com `testApplication` não abrem porta nenhuma (o servidor de teste roda em memória), então esse conflito não existe.

> Erro comum: `Address already in use` / `Port already bound`. Outra instância ficou rodando (um `gradlew run` esquecido em outro terminal, o dev mode do Quarkus). Encontre o processo com `lsof -i :8080` e encerre, ou mude a porta.

📖 Ref. Quarkus — Getting started, seção de testes (porta 8081): https://quarkus.io/guides/getting-started#testing

3.7 Modo de desenvolvimento

- Quarkus: `mvn quarkus:dev` recompila ao salvar e aplica a mudança na próxima requisição (live coding), sem reiniciar à mão. O dev mode também oferece testes contínuos e a Dev UI.
- Ktor: o modo de desenvolvimento (`-Pio.ktor.development=true` no Gradle, ou `ktor { development = true }` no arquivo) liga stack traces detalhados e o recarregamento automático de classes, que depende de configurar os caminhos observados. Sem isso, o ciclo é parar e rodar `./gradlew run` de novo.

Nenhum dos dois modos é para produção: ambos custam desempenho e expõem detalhes internos nas respostas de erro.

📖 Ref. Ktor — Development mode: https://ktor.io/docs/server-development-mode.html

📖 Ref. Quarkus — Continuous testing: https://quarkus.io/guides/continuous-testing

3.8 Resumo do capítulo

Pergunta	Ktor	Quarkus
Onde se configura	código (`embeddedServer`) ou `application.conf` (`EngineMain`)	`application.properties`
Variável de ambiente	`${?PORT}` no HOCON, ou `System.getenv`	automática: `QUARKUS_HTTP_PORT`
Ambientes	propriedade própria + `when` , ou `-config=`	perfis `%dev` , `%test` , `%prod`
Porta padrão	`80` em `embeddedServer` ; o exemplo usa `8080`	`8080` ; testes em `8081`
Host padrão	`0.0.0.0`	`localhost` (dev/test), `0.0.0.0` (prod)

---

4. Kotlin para quem sabe Java

A turma programa em Java. Este capítulo cobre o Kotlin que aparece no código da aula, sempre com o paralelo em Java e o erro que costuma aparecer. Todos os trechos compilam com Kotlin 2.4.10; as saídas mostradas foram obtidas rodando o código.

4.1 `val` , `var` e inferência de tipo

```
val titulo = "Estudar Ktor"   // não pode ser reatribuído; tipo inferido: String
var proximoId = 3             // pode ser reatribuído; tipo inferido: Int
val porta: Int = 8080         // tipo explícito, quando ajuda a leitura
```

Kotlin para quem conhece Java: `val` ≈ variável `final` ; `var` ≈ variável comum. A inferência lembra o `var` do Java 10+, mas vale também para propriedades e retornos de função.

A regra da casa é começar com `val` e trocar para `var` só quando a reatribuição for necessária. A IDE avisa quando um `var` nunca é reatribuído.

O ponto que confunde: `val` impede reatribuir a referência, não mudar o objeto.

```
val tarefas = mutableListOf(Tarefa(1, "Estudar Ktor"))
tarefas.add(Tarefa(2, "Estudar Quarkus"))   // permitido: a lista é mutável
tarefas = mutableListOf()                   // erro de compilação: val não pode ser
        reatribuído
```

As coleções do Kotlin separam leitura de escrita no tipo: `List<T>` só tem operações de leitura; `MutableList<T>` acrescenta `add` , `remove` e afins. `listOf(...)` cria uma `List` ;

`mutableListOf(...)` cria uma `MutableList`.

> Erro comum: achar que devolver `List` protege a lista interna. No `RepositorioEmMemoria` do exemplo, `override fun listar(): List<Tarefa> = tarefas` devolve a própria `MutableList`, só com um tipo mais restrito. Quem recebe não vê `add`, mas um cast recupera o acesso: `(repo.listar() as MutableList).add(...)` alterou o repositório no teste. `List` é uma interface de leitura, não uma cópia. Se o chamador não for de confiança, devolva `tarefas.toList()`. (Em Java acontece o mesmo ao devolver um `ArrayList` tipado como `List`; lá, a proteção é `List.copyOf(...)`.)

📖 Ref. Kotlin — Basic syntax (variáveis): https://kotlinlang.org/docs/basic-syntax.html

📖 Ref. Kotlin — Coleções, de Java para Kotlin: https://kotlinlang.org/docs/java-to-kotlin-collections-guide.html

4.2 Null-safety

Em Kotlin, a possibilidade de `null` faz parte do tipo:

```
val a: String = "texto"     // nunca é null
val b: String? = null       // pode ser null
a.length                    // ok
b.length                    // erro de compilação: b pode ser null
```

As ferramentas para lidar com `String?`:

Operador	Exemplo	Resultado
chamada segura `?.`	`b?.length`	o tamanho, ou `null` se `b` for `null`
elvis `?:`	`b?.length ?: 0`	o tamanho, ou `0`
verificação com `if`	`if (b != null) b.length`	dentro do `if`, `b` é tratado como `String` (smart cast)
asserção `!!`	`b!!.length`	o tamanho, ou `NullPointerException`

Kotlin para quem conhece Java: é a checagem que o `Optional` e as anotações `@Nullable` tentam oferecer, só que feita pelo compilador em todo o código, sem objeto extra.

No código da aula, o null-safety aparece na leitura de parâmetros. `call.parameters["id"]` devolve `String?`, e a conversão encadeia operadores seguros:

```kotlin
val id: Int? = call.parameters["id"]?.toIntOrNull()
val texto: String? = null
println(texto?.length ?: -1)          // -1
println("42".toIntOrNull()?.plus(1))  // 43
```

A fronteira com Java é o ponto frágil. Um método Java sem anotação de nulidade devolve um tipo de plataforma (aparece como `String!` nas mensagens da IDE): o Kotlin não sabe se pode ser `null` e deixa você tratar como quiser. Se você tratar como `String` e vier `null`, a `NullPointerException` volta.

> Erro comum: resolver todo aviso com `!!`. `!!` transforma um erro de compilação em `NullPointerException` em produção. Cada `!!` deveria vir com um motivo claro para o valor nunca ser `null`. Na dúvida, `?.` com `?:`, ou um `if` que responde `400`.

📖 Ref. Kotlin — Null safety: https://kotlinlang.org/docs/null-safety.html

📖 Ref. Kotlin — Nulidade, de Java para Kotlin: https://kotlinlang.org/docs/java-to-kotlin-nullability-guide.html

4.3 Funções: nível superior, corpo de expressão e `Unit`

Três diferenças em relação a métodos Java:

```kotlin
// 1. Função de nível superior: fora de qualquer classe
fun main() { /* ... */ }

// 2. Corpo de expressão: "= expressão" no lugar de { return expressão }
fun dobro(x: Int): Int = x * 2
fun dobroInferido(x: Int) = x * 2      // o tipo de retorno é inferido

// 3. Unit: a função não devolve nada útil
fun registrar(msg: String): Unit { println(msg) }
fun registrarCurto(msg: String) { println(msg) }   // ": Unit" pode ser omitido
```

Kotlin para quem conhece Java: função de nível superior é compilada como método `static` numa classe gerada com o nome do arquivo mais `Kt`. As funções de `Aplicacao.kt` viram métodos estáticos de `AplicacaoKt`, e é por isso que o `build.gradle.kts` aponta `mainClass` para `br.ufrn.exemplo.tarefas.AplicacaoKt`. `Unit` corresponde ao `void`, com uma diferença: é um tipo de verdade, com um único valor, e pode aparecer em genéricos (`() -> Unit`).

📖 Ref. Kotlin — Functions: https://kotlinlang.org/docs/functions.html

📖 Ref. Kotlin — Chamando Kotlin a partir de Java (classe `ArquivoKt`): https://kotlinlang.org/docs/java-to-kotlin-interop.html

4.4 Lambdas e tipos de função

Uma lambda é um bloco de código tratado como valor: pode ser guardada numa variável, passada como argumento ou devolvida por uma função.

```
val dobro: (Int) -> Int = { x -> x * 2 }
val dobroCurto: (Int) -> Int = { it * 2 }   // parâmetro único: pode usar "it"
val saudar: () -> Unit = { println("oi") }

dobro(21)      // 42
saudar()       // imprime "oi"
```

A sintaxe da lambda: chaves, parâmetros antes da seta, corpo depois. A última expressão do corpo é o valor devolvido; não se escreve `return`.

O tipo de uma função se escreve `(Entradas) -> Saída`:

Tipo Kotlin	Recebe	Devolve	Parecido em Java
`() -> Unit`	nada	nada útil	`Runnable`
`() -> T`	nada	`T`	`Supplier<T>`
`(A) -> B`	`A`	`B`	`Function<A, B>`
`(A) -> Unit`	`A`	nada útil	`Consumer<A>`
`(A, B) -> C`	`A`, `B`	`C`	`BiFunction<A, B, C>`
`(T) -> Boolean`	`T`	`Boolean`	`Predicate<T>`

Kotlin para quem conhece Java: em Java, uma lambda só existe como implementação de uma interface funcional, e cada formato pede uma interface (`Runnable`, `Function`, `BiFunction`...). Em Kotlin, funções têm tipo próprio; não é preciso escolher uma interface.

Nas duas direções da interoperabilidade, a conversão é automática. Uma lambda Kotlin serve onde Java espera uma interface funcional (conversão SAM), e interfaces Kotlin declaradas com `fun interface` aceitam lambda do mesmo jeito:

```kotlin
val r = Runnable { println("rodando") }     // interface Java
r.run()

fun interface Validador { fun valida(s: String): Boolean }
val naoVazio = Validador { it.isNotBlank() }
naoVazio.valida(" ")                         // false
```

Uma lambda enxerga e pode alterar variáveis do escopo onde foi criada (closure). Em Java, variáveis capturadas precisam ser efetivamente `final`; em Kotlin, um `var` capturado pode ser alterado dentro da lambda.

> Erro comum: usar `return` dentro de uma lambda para sair dela. Um `return` sem rótulo dentro de lambda tenta sair da função que a contém, e o compilador só permite isso quando a função que recebe a lambda é `inline`. Para sair só da lambda, use o rótulo com o nome da função que a recebeu: `return@get`, `return@post`, `return@forEach`. É o `return@get` da seção 2.3.

📖 Ref. Kotlin — Higher-order functions and lambdas: https://kotlinlang.org/docs/lambdas.html

📖 Ref. Kotlin — Functional (SAM) interfaces: https://kotlinlang.org/docs/fun-interfaces.html

4.5 Trailing lambda

Quando o último parâmetro de uma função é uma função, a lambda pode ser escrita fora dos parênteses. Se ela for o único argumento, os parênteses somem:

```kotlin
fun repetir(vezes: Int = 1, acao: (Int) -> Unit) {
  for (i in 0 until vezes) acao(i)
}

repetir(2, { i -> println("volta $i") })   // forma com parênteses
repetir(2) { i -> println("volta $i") }    // trailing lambda
repetir { println("uma vez, volta $it") }  // vezes usa o padrão; só a lambda
```

Saída das duas últimas chamadas: `volta 0`, `volta 1`, `uma vez, volta 0`.

É isso que faz o código Ktor parecer ter palavras-chave novas. Nenhuma delas é da linguagem; todas são funções recebendo uma lambda como último argumento:

```
embeddedServer(CIO, port = 8080) { modulo() }   // (factory, port, ..., module)
install(ContentNegotiation) { json() }          // (plugin, configure)
routing { /* ... */ }                           // (configuration)
get("/tarefas") { /* ... */ }                   // (path, body)
```

Kotlin para quem conhece Java: é o equivalente a `embeddedServer(CIO, 8080, app -> modulo(app))`, com a lambda visualmente fora da chamada.

> Erro comum: a lambda não é o último parâmetro. A sintaxe só vale para o último. Numa função `fun f(acao: () -> Unit, vezes: Int)`, `f { }` não compila; é preciso `f({ }, 3)`. Ao desenhar uma função que recebe lambda, coloque a lambda por último.

📖 Ref. Kotlin — Passing trailing lambdas: https://kotlinlang.org/docs/lambdas.html#passing-trailing-lambdas

4.6 Funções de extensão, lambda com receptor e type-safe builders

Este é o recurso que sustenta o DSL do Ktor. Vale ir em três passos.

Passo 1, função de extensão. Kotlin permite declarar uma função "de fora" que é chamada como se fosse método do tipo:

```
fun String.gritar(): String = this.uppercase() + "!"

"oi".gritar()   // "OI!"
```

Dentro da função, `this` é o objeto sobre o qual ela foi chamada (o receptor). O tipo `String` não foi alterado; a extensão é resolvida em compilação.

Kotlin para quem conhece Java: uma extensão compila para um método estático cujo primeiro parâmetro é o receptor. Se a extensão estiver no arquivo `Texto.kt`, `"oi".gritar()` equivale a `TextoKt.gritar("oi")`, como um método utilitário `StringUtils.gritar(s)`, com sintaxe de método.

O exemplo da aula já usa extensões: `fun Application.modulo()` e `fun Application.rotas()`. Dentro delas, `this` é a `Application`, e por isso `install(...)` e `routing { }` são chamados sem prefixo.

Passo 2, lambda com receptor. Assim como uma função pode ter receptor, um tipo de função também pode: `T.() -> Unit` é uma função que roda com um `T` como `this`.

```
val configurar: StringBuilder.() -> Unit = {
  append("Estudar ")   // this.append(...), this é o StringBuilder
  append("Ktor")
}
val sb = StringBuilder()
sb.configurar()
println(sb)            // Estudar Ktor
```

A biblioteca padrão usa isso em `apply`: `StringBuilder().apply { append("x") }` cria o objeto, roda o bloco com ele como `this` e o devolve.

No Ktor, as assinaturas reais (versão 3.5.2) mostram o receptor de cada bloco:

```
fun Application.routing(configuration: Routing.() -> Unit): RoutingRoot
fun Route.route(path: String, build: Route.() -> Unit): Route
fun Route.get(path: String, body: RoutingHandler): Route
typealias RoutingHandler = suspend RoutingContext.() -> Unit
```

Leitura: dentro de `routing { }`, `this` é um `Routing`; dentro de `route("/x") { }`, um `Route`; dentro do handler de `get { }`, um `RoutingContext`, e `call` é uma propriedade dele. Por isso `call` aparece "do nada" nos handlers.

Passo 3, type-safe builder. Juntando os dois recursos, dá para construir um DSL. Um exemplo mínimo que gera HTML:

```
class Html {
  private val partes = mutableListOf<String>()
  fun p(texto: String) { partes += "<p>$texto</p>" }
  fun ul(bloco: Lista.() -> Unit) { partes += Lista().apply(bloco).render() }
  fun render() = partes.joinToString("")
}

class Lista {
  private val itens = mutableListOf<String>()
  fun li(texto: String) { itens += "<li>$texto</li>" }
  fun render() = "<ul>${itens.joinToString("")}</ul>"
}

fun html(bloco: Html.() -> Unit): String = Html().apply(bloco).render()
```

Uso:

```
val pagina = html {
  p("Tarefas")
  ul {
    li("Estudar Ktor")
    li("Estudar Quarkus")
  }
}
// <p>Tarefas</p><ul><li>Estudar Ktor</li><li>Estudar Quarkus</li></ul>
```

O que acontece, em ordem:

1. `html { ... }` chama a função `html` com a lambda como argumento (trailing lambda).
2. `html` cria um `Html` e roda a lambda com ele como `this` (`apply`).
3. Dentro da lambda, `p(...)` é `this.p(...)`, e `ul { ... }` é `this.ul(...)`.
4. `ul` cria uma `Lista` e roda o bloco interno com ela como `this`, então `li(...)` é `lista.li(...)`.
5. O compilador verifica tudo: `li` fora de `ul` não compila, porque `Html` não tem `li`.

`routing { route("/tarefas") { get { } } }` funciona exatamente assim.

> Erro comum: chamar, num bloco interno, uma função do bloco de fora. Dentro de `ul { }`, o `this` de `html { }` continua acessível. `ul { li("a"); p("x") }` compila, e o `<p>` vai parar fora da lista: no teste, a saída foi `<p>x</p><ul><li>a</li></ul>`. DSLs maiores marcam as classes com uma anotação `@DslMarker` para que o compilador proíba esse acesso implícito ao receptor externo. No Ktor, um sintoma parecido é declarar `get { }` dentro do handler de outra rota por engano.

📖 Ref. Kotlin — Extensions: https://kotlinlang.org/docs/extensions.html

📖 Ref. Kotlin — Function literals with receiver: https://kotlinlang.org/docs/lambdas.html#function-literals-with-receiver

📖 Ref. Kotlin — Type-safe builders: https://kotlinlang.org/docs/type-safe-builders.html

📖 Ref. Kotlin — Scope functions (`apply`, `also`, `let`): https://kotlinlang.org/docs/scope-functions.html

4.7 Argumentos nomeados e valores padrão

```kotlin
data class Tarefa(val id: Int, val titulo: String, val feita: Boolean = false)

Tarefa(1, "Estudar")                          // feita = false (valor padrão)
Tarefa(1, "Estudar", true)
Tarefa(id = 1, titulo = "Estudar", feita = true)
embeddedServer(CIO, port = 8080, host = "0.0.0.0") { modulo() }
```

Valor padrão (`= false`) torna o argumento opcional. Argumento nomeado (`port = 8080`) deixa a chamada legível e permite pular parâmetros opcionais no meio da lista.

Kotlin para quem conhece Java: substitui a pilha de sobrecargas (`Tarefa(int, String)` , `Tarefa(int, String, boolean)`) e boa parte dos builders. Para uma função com padrões ser chamada de Java com menos argumentos, anote com `@JvmOverloads` : o compilador gera as sobrecargas.

> Erro comum: renomear um parâmetro achando que é mudança interna. Com argumentos nomeados, o nome do parâmetro faz parte da API. Renomear `titulo` para `nome` quebra todo chamador que escreveu `titulo = ...` .

📖 Ref. Kotlin — Named arguments e default values: https://kotlinlang.org/docs/functions.html#named-arguments

4.8 `data class` × `record`

```kotlin
data class Tarefa(val id: Int, val titulo: String, val feita: Boolean = false)
```

```java
public record Tarefa(int id, String titulo, boolean feita) {}
```

Os dois declaram uma classe de dados em uma linha e geram `equals` , `hashCode` e `toString` a partir dos campos. As diferenças:

	`data class`	`record`
Campos	`val` (leitura) ou `var` (leitura e escrita)	sempre finais
Acesso	propriedade: `t.titulo`	método: `t.titulo()`
Valores padrão	sim (`= false`)	não; exige construtor extra

	`data class`	`record`
Cópia com alteração	`copy(...)` gerado	não gerado
Desestruturação	`val (id, titulo) = t`	via padrões de record (`instanceof` / `switch`)
Herança	pode implementar interfaces e estender classe aberta	só implementa interfaces

O `copy` é a peça mais útil. Como os campos são `val`, "alterar" uma tarefa é criar outra:

```
val t = Tarefa(1, "Estudar Ktor")
val concluida = t.copy(feita = true)
println(t)          // Tarefa(id=1, titulo=Estudar Ktor, feita=false)
println(concluida)  // Tarefa(id=1, titulo=Estudar Ktor, feita=true)
println(t == Tarefa(1, "Estudar Ktor", false))   // true: igualdade por valor
```

Só as propriedades do construtor primário entram em `equals`, `hashCode`, `toString`, `copy` e desestruturação. Uma propriedade declarada no corpo da classe fica de fora.

> Erro comum: `var` em data class usada como chave de `HashMap` ou elemento de `HashSet`. O `hashCode` depende dos campos; alterar um `var` depois de inserir faz o objeto "sumir" do conjunto (continua lá, no balde errado). Prefira `val` e `copy`.

📖 Ref. Kotlin — Data classes: https://kotlinlang.org/docs/data-classes.html

📖 Ref. Java 25 — Record classes: https://docs.oracle.com/en/java/javase/25/language/records.html

4.9 Propriedades delegadas (`by`)

O exemplo tem esta linha:

```
val repositorio by inject<RepositorioDeTarefas>()
```

`by` diz que ler a propriedade é delegado a outro objeto, o delegate. O compilador reescreve o código mais ou menos assim:

```
// o que você escreve
val repositorio by inject<RepositorioDeTarefas>()

// o que o compilador gera (simplificado)
private val repositorio$delegate = inject<RepositorioDeTarefas>()
val repositorio: RepositorioDeTarefas
  get() = repositorio$delegate.getValue(this, ::repositorio)
```

Qualquer objeto com um método `operator fun getValue(...)` (e `setValue`, para `var`) serve de delegate. Um delegate próprio mostra o mecanismo:

```
class Registrando<T>(private var valor: T) {
  operator fun getValue(dono: Any?, prop: KProperty<*>): T {
    println("lendo ${prop.name}")
    return valor
  }
  operator fun setValue(dono: Any?, prop: KProperty<*>, novo: T) {
    println("${prop.name}: $valor -> $novo")
    valor = novo
  }
}

var contador by Registrando(0)
contador += 1
println(contador)
```

Saída:

```
lendo contador
contador: 0 -> 1
lendo contador
1
```

Kotlin para quem conhece Java: é um getter e um setter cujo corpo mora em outro objeto reutilizável. Em Java, você escreveria o campo privado, o getter que chama o objeto auxiliar e repetiria isso em cada classe.

Delegates que aparecem no curso:

- `by lazy { ... }`: calcula no primeiro acesso e guarda. Por padrão, é seguro para várias threads.

  ```
  val pesado by lazy { println("calculando"); 42 }
  println("antes"); println(pesado); println(pesado)
  // antes / calculando / 42 / 42
  ```

- `by inject<T>()` (Koin): no Koin 4.2.2, `inject` é literalmente `lazy { get<T>(...) }`. A implementação só é buscada no primeiro acesso a `repositorio`, não na linha da declaração.
- `val id: Int by call.parameters` (Ktor): lê o parâmetro `id` e converte para `Int`. Se a conversão falhar, o Ktor responde `400` (conferido com `/abc`).
- `by remember { mutableStateOf(...) }`: no Compose, na disciplina de Móveis.

> Erro comum: esquecer o import do `getValue`. Delegates definidos como extensão precisam ser importados. Sem `import io.ktor.server.util.getValue`, a linha `val id: Int by call.parameters` não compila, com a mensagem `Type 'Parameters' has no method 'getValue(Nothing?, KProperty0<*>)', so it cannot serve as a delegate`. A IDE sugere o import; aceite.

📖 Ref. Kotlin — Delegated properties: https://kotlinlang.org/docs/delegated-properties.html

📖 Ref. Kotlin — Regras de tradução de delegates: https://kotlinlang.org/docs/delegated-properties.html#translation-rules-for-delegated-properties

4.10 Genéricos: type erasure, `inline` + `reified` e `Class<T>`

Na JVM, os argumentos de tipo genérico não existem em tempo de execução (type erasure). `List<String>` e `List<Int>` são a mesma classe para a JVM:

```
fun <T> nomeDaClasse(lista: List<T>): String = lista::class.simpleName ?: "?"
nomeDaClasse(listOf("a"))   // "SingletonList": nenhuma menção a String
```

Por isso APIs Java que precisam saber o tipo pedem a classe como argumento:

```
NovaTarefa nova = mapper.readValue(json, NovaTarefa.class);
List<Tarefa> lista = mapper.readValue(json, new TypeReference<List<Tarefa>>() {});
```

O `TypeReference` com `{}` é um truque: a subclasse anônima guarda o tipo completo na assinatura da classe, onde ele sobrevive ao apagamento.

Kotlin oferece outra saída, para funções `inline` com parâmetro de tipo `reified`:

```
inline fun <reified T> nomeDoTipo(): String = T::class.simpleName ?: "?"

nomeDoTipo<NovaTarefa>()   // "NovaTarefa"
```

- `inline` faz o compilador copiar o corpo da função para cada local de chamada.
- `reified` só é permitido em função `inline`: como o corpo é copiado, o compilador cola ali o tipo concreto usado naquela chamada. Dentro da função, `T::class` passa a funcionar.

É assim que o Ktor dispensa o `.class`. As assinaturas na versão 3.5.2:

```
suspend inline fun <reified T : Any> ApplicationCall.receive(): T =
  receiveNullable(typeInfo<T>())

suspend inline fun <reified T : Any> ApplicationCall.respond(message: T) {
  respond(message, typeInfo<T>())
}
```

`typeInfo<T>()` captura o tipo completo, inclusive argumentos (`List<Tarefa>`, não só `List`). Com isso, `call.respond(repositorio.listar())` sabe que precisa de um serializador de lista de `Tarefa`.

	Java	**Kotlin**
Desserializar um tipo simples	`readValue(json, NovaTarefa.class)`	`call.receive<NovaTarefa>()`
Tipo com argumentos	`new TypeReference<List<Tarefa>>() {}`	`receive<List<Tarefa>>()`
Custo	nenhum	corpo duplicado em cada chamada (por isso, funções pequenas)

> Erro comum: chamar função `reified` a partir de função genérica comum. `fun <T : Any> ler(call: ApplicationCall): T = call.receive<T>()` não compila: dentro de `ler`, `T` já foi apagado, e o compilador responde `Cannot use 'T' as reified type parameter. Use a class instead.` A solução é tornar `ler` também `inline` com `reified T`, ou passar a classe (`KClass<T>`) explicitamente.

📖 Ref. Kotlin — Generics, type erasure: https://kotlinlang.org/docs/generics.html#type-erasure

📖 Ref. Kotlin — Reified type parameters: https://kotlinlang.org/docs/inline-functions.html#reified-type-parameters

4.11 Miudezas que aparecem no código

Kotlin	Java	Observação
`class A : B, C`	`class A extends B implements C`	um só `:` para os dois
`override fun listar()`	`@Override public List<...> listar()`	`override` é obrigatório
`"id=$id, titulo=${t.titulo}"`	`"id=" + id + ", titulo=" + t.titulo()`	string template
`Tarefa(1, "X")`	`new Tarefa(1, "X")`	sem `new`
`x.also { lista.add(it) }`	`lista.add(x); return x;`	executa algo e devolve o próprio `x`
`when (x) { 1 -> "um"; else -> "?" }`	`switch` com `->`	`when` é expressão
`lista.filter { it.feita }`	`lista.stream().filter(Tarefa::feita).toList()`	funções direto na coleção
`mapOf("status" to "no ar")`	`Map.of("status", "no ar")`	`to` cria um par

A classe e seus membros são `public` e `final` por padrão em Kotlin. Para permitir herança, marque com `open`.

📖 Ref. Kotlin — Comparison to Java: https://kotlinlang.org/docs/comparison-to-java.html

---

5. JSON: como o objeto vira resposta (e a requisição vira objeto)

Uma rota devolve um objeto (`List<Tarefa>`) e o cliente recebe texto JSON. No `POST`, o caminho é o inverso. Serializar é converter objeto em texto; desserializar é converter texto em objeto. Os dois stacks

fazem isso com bibliotecas de filosofias diferentes, e as diferenças aparecem justamente quando o JSON que chega não é o esperado.

5.1 Duas estratégias: reflexão × código gerado

Para serializar `Tarefa`, a biblioteca precisa saber quais campos a classe tem.

- Jackson (Quarkus) descobre em tempo de execução, por reflexão: inspeciona a classe, encontra os componentes do record (ou getters e campos) e monta o JSON.
- kotlinx.serialization (Ktor) descobre em tempo de compilação: um plugin do compilador Kotlin gera, para cada classe `@Serializable`, um serializador com o código de leitura e escrita. Em execução, não há reflexão.

	kotlinx.serialization	**Jackson**
Como conhece a classe	plugin gera serializador na compilação	reflexão em execução
Marcação exigida	`@Serializable` em cada classe	nenhuma (convenções)
Classe sem marcação	erro	funciona
Linguagens	Kotlin (inclui multiplataforma)	Java e qualquer linguagem JVM
Ecossistema	JSON, CBOR, ProtoBuf	JSON, XML, YAML, CSV e muitos módulos

O Quarkus também tem uma otimização que gera serializadores Jackson no build, sem reflexão (`quarkus.rest.jackson.optimization.enable-reflection-free-serializers=true`). Na versão atual, ela é marcada como prévia de tecnologia e vem desligada.

📖 Ref. Kotlin — Serialization: https://kotlinlang.org/docs/serialization.html

📖 Ref. Quarkus — Writing JSON REST services: https://quarkus.io/guides/rest-json

5.2 kotlinx.serialization no Ktor

Três peças precisam estar presentes:

```
// build.gradle.kts
plugins {
  kotlin("plugin.serialization") version "2.4.10"   // 1. o plugin do compilador
}
dependencies {
  implementation("io.ktor:ktor-server-content-negotiation:3.5.2")  // 2. o plugin do Ktor
  implementation("io.ktor:ktor-serialization-kotlinx-json:3.5.2")  // 3. o conversor JSON
}
```

```
@Serializable
data class Tarefa(val id: Int, val titulo: String, val feita: Boolean = false)

fun Application.modulo() {
  install(ContentNegotiation) { json() }
  // ...
}
```

A anotação sozinha não faz nada; quem gera o serializador é o plugin do compilador. A biblioteca também pode ser usada fora do Ktor:

```
val json = Json.encodeToString(Tarefa(1, "Ler"))          // {"id":1,"titulo":"Ler"}
val nova = Json.decodeFromString<NovaTarefa>("""{"titulo":"Ler"}""")
```

Repare na saída: `feita` não apareceu. Esse detalhe é assunto da seção 5.4.

> Erro comum: `@Serializable` sem o plugin no `build.gradle.kts`. A anotação compila, mas o serializador não é gerado, e a falha só aparece quando a rota tenta serializar. Ao ver erro dizendo que não há serializador para a classe, confira primeiro a linha `kotlin("plugin.serialization")`, com a mesma versão do Kotlin.

📖 Ref. Ktor — Content negotiation and serialization: https://ktor.io/docs/server-serialization.html

📖 Ref. kotlinx.serialization — Basic serialization: https://github.com/Kotlin/kotlinx.serialization/blob/master/docs/basic-serialization.md

5.3 Negociação de conteúdo no Ktor

`ContentNegotiation` faz a ponte entre HTTP e os conversores:

- Na resposta, `call.respond(objeto)` olha o `Accept` da requisição e escolhe um conversor registrado para aquele formato. Sem `Accept`, usa o que houver.
- Na requisição, `call.receive<T>()` olha o `Content-Type` e escolhe o conversor para ler o corpo.

Como só JSON foi registrado:

Requisição	Resultado conferido
`GET /tarefas` sem `Accept` ou com `Accept: application/json`	`200`, JSON
`GET /tarefas` com `Accept: application/xml`	`406 Not Acceptable`

Requisição	Resultado conferido
`POST /tarefas` com `Content-Type: application/json`	`201`
`POST /tarefas` com `Content-Type: text/plain` ou sem cabeçalho (curl `-d`)	`415 Unsupported Media Type`

Registrar outro formato é mais uma linha no mesmo bloco (`xml()`, `cbor()`), com a dependência correspondente.

5.4 O `Json` padrão do Ktor e o erro do `prettyPrint`

`json()` sem argumentos não usa as configurações padrão do kotlinx.serialization. Usa um objeto `DefaultJson` definido pelo Ktor (código-fonte da versão 3.5.2):

```
public val DefaultJson: Json = Json {
  encodeDefaults = true
  isLenient = true
  allowSpecialFloatingPointValues = true
  allowStructuredMapKeys = true
  prettyPrint = false
  useArrayPolymorphism = false
}
```

A linha que importa é `encodeDefaults = true`. No kotlinx.serialization, por padrão, propriedades com valor igual ao padrão não são escritas (o raciocínio é que o outro lado vai preencher o padrão ao ler). Com o `DefaultJson` do Ktor, são:

```
Json padrão         {"id":1,"titulo":"Ler"}
DefaultJson (Ktor)  {"id":1,"titulo":"Ler","feita":false}
```

Opções do `Json` que costumam ser ajustadas:

Opção	Padrão do kotlinx	No `DefaultJson`	Efeito
`encodeDefaults`	`false`	`true`	escreve campos com valor padrão
`ignoreUnknownKeys`	`false`	`false`	aceita chaves que a classe não tem
`explicitNulls`	`true`	`true`	escreve `null` explicitamente
`prettyPrint`	`false`	`false`	JSON indentado

Opção	Padrão do kotlinx	No `DefaultJson`	Efeito
`isLenient`	`false`	`true`	aceita JSON fora do padrão (ex.: strings sem aspas)

> Erro comum: `json(Json { prettyPrint = true })` e o campo `feita` some. `Json { }` parte das configurações do kotlinx, não do `DefaultJson`. Ao pedir JSON indentado assim, `encodeDefaults` volta a `false`, e toda tarefa pendente perde o campo `feita` na resposta (conferido). O cliente que esperava `feita` quebra. Para ajustar uma opção mantendo as demais do Ktor, parta do `DefaultJson`:
>
> ```
> install(ContentNegotiation) {
>   json(Json(DefaultJson) { prettyPrint = true })
> }
> ```

📖 Ref. kotlinx.serialization — Json configuration: https://github.com/Kotlin/kotlinx.serialization/blob/master/docs/json.md

5.5 Jackson no Quarkus

No Quarkus, a extensão `quarkus-rest-jackson` registra o Jackson como leitor e escritor de JSON. Não há `install`: a presença da dependência basta, e métodos que devolvem objetos passam a produzir `application/json` (seção 2.5).

Records funcionam sem configuração: o Jackson usa os componentes do record para escrever o JSON e o construtor canônico para ler. Para classes comuns, a documentação do Quarkus lembra que o Jackson precisa de um construtor sem argumentos.

O `ObjectMapper` que o Quarkus fornece difere do Jackson puro em dois pontos:

- Ignora propriedades desconhecidas (`FAIL_ON_UNKNOWN_PROPERTIES` desligado). Para voltar ao comportamento estrito: `quarkus.jackson.fail-on-unknown-properties=true`.
- Escreve datas em ISO-8601 (`WRITE_DATES_AS_TIMESTAMPS` desligado), em vez de números.

Para ajustes que não têm propriedade pronta, cria-se um bean que implementa `ObjectMapperCustomizer`:

```java
@Singleton
public class ConfiguracaoJackson implements ObjectMapperCustomizer {
  @Override
  public void customize(ObjectMapper mapper) {
    mapper.enable(SerializationFeature.INDENT_OUTPUT);
  }
}
```

Essa via preserva as configurações que o Quarkus e as extensões aplicam. Declarar um `ObjectMapper` do zero exige reaplicar todos os customizadores à mão.

📖 Ref. Quarkus — Configuring JSON support (Jackson): https://quarkus.io/guides/rest-json#jackson

5.6 Mesmo JSON, regras diferentes

O caminho feliz é idêntico nos dois exemplos. As diferenças estão no JSON que chega diferente do esperado. Resultados conferidos com `POST /tarefas` e `Content-Type: application/json`:

Corpo enviado	Ktor (kotlinx)	Quarkus (Jackson)
`{"titulo":"Ler"}`	`201`	`201`
`{"titulo":"Ler","prioridade":1}`	`400` : chave desconhecida	`201` : a chave é ignorada
`{}`	`400` : campo obrigatório `titulo` ausente	`201` , tarefa criada com `"titulo":null`
`{"titulo":null}`	`400` : `null` onde se espera `String`	`201` , com `"titulo":null`
`{"titulo":` (malformado)	`400`	`400`
corpo vazio	`400`	`500` : `NullPointerException` no método

Por que o Ktor é mais rígido: `NovaTarefa(val titulo: String)` declara `titulo` como não nulo e sem padrão, e o serializador gerado respeita o tipo. O null-safety do Kotlin chega até a fronteira HTTP. No Java, `String titulo` aceita `null` , e o Jackson preenche o que faltar com `null` .

Nenhum dos dois comportamentos está errado; são padrões diferentes. O que não pode acontecer é o projeto descobrir a diferença em produção, com tarefas sem título no banco. Os caminhos:

- No Ktor, se chaves extras devem ser aceitas (clientes mais novos mandando campos a mais), configure `ignoreUnknownKeys = true` partindo do `DefaultJson` .

- No Quarkus, validar a entrada: com a extensão Hibernate Validator, anotar o campo (`@NotBlank String titulo`) e o parâmetro (`@Valid NovaTarefa nova`) faz a requisição inválida receber `400` .
- No Ktor, o plugin `RequestValidation` cumpre o papel de regras além do tipo (título em branco, tamanho máximo).
- Nos dois, a mensagem de erro para o cliente é assunto do capítulo 6.

📖 Ref. Quarkus — Validation with Hibernate Validator: https://quarkus.io/guides/validation

📖 Ref. Ktor — Request validation: https://ktor.io/docs/server-request-validation.html

5.7 Nomes de campo, datas e outros ajustes

Nomes diferentes no JSON e no código. Se o contrato usa `snake_case` e o código usa `camelCase` :

```kotlin
@Serializable
data class Tarefa(val id: Int, @SerialName("titulo_da_tarefa") val titulo: String)
```

```java
public record Tarefa(int id, @JsonProperty("titulo_da_tarefa") String titulo) {}
```

Datas. O Jackson do Quarkus serializa `java.time.LocalDate` em ISO-8601 (`"2026-09-14"`) sem configuração. O kotlinx.serialization não traz serializador para `java.time` : ou se escreve um serializador próprio, ou se usa os tipos de `kotlinx-datetime` . Vale decidir isso antes de a primeira data aparecer no modelo.

Campos que não devem sair na resposta. `@Transient` (kotlinx) e `@JsonIgnore` (Jackson) tiram um campo da serialização. Para dados sensíveis, o mais seguro é não tê-los no tipo de resposta: um tipo de saída separado, como `NovaTarefa` separa a entrada.

> Erro comum: devolver a entidade do banco direto na rota. Na parte 2 (21/09), a classe persistida ganha campos que o cliente não deve ver ou que mudam com o esquema do banco. Devolvê-la diretamente amarra o contrato da API ao esquema. Manter tipos de entrada e saída próprios, como `NovaTarefa` e `Tarefa` , evita isso.

📖 Ref. kotlinx.serialization — Basic serialization (`@SerialName` , `@Transient`): https://github.com/Kotlin/kotlinx.serialization/blob/master/docs/basic-serialization.md

---

6. Corpo da requisição e status da resposta

O capítulo 5 tratou da conversão entre JSON e objeto. Este capítulo trata do que acontece em volta dela: como a rota obtém o corpo, como escolhe o status, como informa onde está o recurso criado e como devolve erros que o cliente consegue tratar. É também onde o conteúdo da Sprint 0 (status corretos e erros em `application/problem+json`) vira código.

6.1 Ler o corpo: `receive<T>()` × parâmetro do método

No Ktor, ler o corpo é uma chamada explícita dentro do handler:

```
post {
  val nova = call.receive<NovaTarefa>()   // lê e converte o corpo aqui
  // ...
}
```

No Quarkus, o corpo chega como parâmetro. Um parâmetro sem anotação de parâmetro (`@RestPath`, `@RestQuery`...) é o corpo, já convertido antes de o método começar:

```
@POST
public Response criar(NovaTarefa nova) {   // o Quarkus converteu o corpo antes da chamada
  // ...
}
```

	Ktor	Quarkus
Quando o corpo é lido	na linha do `receive`	antes de o método ser chamado
Como o tipo é informado	`reified T` (seção 4.10)	tipo do parâmetro
Erro de conversão	exceção na linha do `receive`	o método nem é chamado
Ler o corpo como texto	`call.receiveText()`	parâmetro `String`

Kotlin para quem conhece Java: `receive<NovaTarefa>()` corresponde ao `mapper.readValue(corpo, NovaTarefa.class)` que você escreveria num Servlet. O Quarkus esconde essa linha; o Ktor a mantém visível, sem o `.class`.

O corpo de uma requisição é um fluxo de bytes e só pode ser lido uma vez. No Ktor, um segundo `receive` na mesma requisição lança `RequestAlreadyConsumedException`, e o cliente recebe `500` (conferido). Se precisar do corpo em dois lugares, leia uma vez e passe o objeto adiante.

📖 Ref. Ktor — Receiving body contents: https://ktor.io/docs/server-requests.html

📖 Ref. Quarkus REST — Accessing the request body: https://quarkus.io/guides/rest#accessing-the-request-body

6.2 Escolher o status

Sem instrução, os dois respondem `200 OK`. Os outros status precisam ser pedidos:

```
call.respond(tarefa)                              // 200 com corpo
call.respond(HttpStatusCode.Created, criada)      // 201 com corpo
call.respond(HttpStatusCode.NoContent)            // 204 sem corpo
```

No Quarkus, há três formas:

```
// 1. Tipo de retorno: objeto → 200; void → 204
@GET @Path("/{id}")
public Tarefa buscar(@RestPath int id) { /* ... */ }

@DELETE @Path("/{id}")
public void remover(@RestPath int id) { /* ... */ }       // 204 (conferido)

// 2. Response (Jakarta REST): status, cabeçalhos e corpo, sem tipo
return Response.status(Response.Status.CREATED).entity(criada).build();

// 3. RestResponse<T> (Quarkus REST): o mesmo, com o tipo do corpo no retorno
public RestResponse<Tarefa> criar(NovaTarefa nova) {
  return RestResponse.status(RestResponse.Status.CREATED, repositorio.adicionar(nova));
}
```

`RestResponse<Tarefa>` diz no tipo de retorno o que vai no corpo; `Response` aceita qualquer objeto em `entity(...)`. Para um status fixo sem cabeçalhos extras, o Quarkus REST também oferece a anotação `@ResponseStatus(201)` no método.

📖 Ref. Ktor — Sending responses (status): https://ktor.io/docs/server-responses.html

📖 Ref. Quarkus REST — Returning a response body: https://quarkus.io/guides/rest#returning-a-response-body

6.3 `201 Created` com `Location`

A checklist da Sprint 1 pede: `POST` cria e devolve `201` com `Location`. O cabeçalho diz ao cliente onde está o recurso criado; sem ele, a RFC 9110 considera que o recurso é a própria URI da requisição, ou seja, `/tarefas`, o que não é verdade.

```
post {
  val nova = call.receive<NovaTarefa>()
  val criada = repositorio.adicionar(nova)
  call.response.header(HttpHeaders.Location, "/tarefas/${criada.id}")
  call.respond(HttpStatusCode.Created, criada)
}
```

```
@POST
public Response criar(NovaTarefa nova) {
  Tarefa criada = repositorio.adicionar(nova);
  return Response.created(URI.create("/tarefas/" + criada.id()))
      .entity(criada)
      .build();
}
```

`Response.created(...)` já define o status `201`. Uma diferença conferida: o Ktor envia o valor como foi escrito (`Location: /tarefas/3`); o Quarkus resolve a URI relativa contra o endereço do servidor e envia `Location: http://localhost:8081/tarefas/3`. As duas formas são válidas: o cabeçalho aceita referência relativa.

```
# cabeçalhos resumidos
$ curl -i -X POST localhost:8080/tarefas -H 'Content-Type: application/json' -d
'{"titulo":"Ler"}'
HTTP/1.1 201 Created
Location: /tarefas/3
Content-Type: application/json

{"id":3,"titulo":"Ler","feita":false}
```

📖 Ref. MDN — Location (PT): https://developer.mozilla.org/pt-BR/docs/Web/HTTP/Reference/Headers/Location

6.4 `DELETE`, `204` e idempotência na prática

```
delete("/{id}") {
  val id = call.parameters["id"]?.toIntOrNull() ?: throw BadRequestException("id deve ser um
        número inteiro")

  if (!repositorio.remover(id)) throw TarefaNaoEncontrada(id)

  call.respond(HttpStatusCode.NoContent)
}
```

```
@DELETE
@Path("/{id}")
public void remover(@RestPath int id) {
  if (!repositorio.remover(id)) throw new TarefaNaoEncontrada(id);
}
```

Conferido nos dois: o primeiro `DELETE /tarefas/1` responde `204`; o segundo, `404`. Isso não quebra a idempotência (seção 1.3): o estado do servidor depois de uma ou de duas chamadas é o mesmo (a tarefa 1 não existe). A resposta pode variar; o efeito, não.

As rotas acima lançam exceções em vez de responder o erro na hora. A próxima seção explica por quê.

📖 Ref. MDN — 204 No Content (PT): https://developer.mozilla.org/pt-BR/docs/Web/HTTP/Reference/Status/204

6.5 Erros como exceções, tratados num lugar só

Na seção 2.3, a rota respondia `400` e `404` no meio do código, com `return@get` depois de cada resposta. Com três ou quatro rotas, isso espalha a decisão de formato de erro pelo projeto. A alternativa é separar:

- o código que detecta o problema lança uma exceção com significado (`TarefaNaoEncontrada`);
- um tratador central, na borda HTTP, decide status e corpo.

```
class TarefaNaoEncontrada(val id: Int) : RuntimeException("Tarefa $id não encontrada")


get("/{id}") {
  val id = call.parameters["id"]?.toIntOrNull() ?: throw BadRequestException("id deve ser um
        número inteiro")

  val tarefa = repositorio.buscar(id) ?: throw TarefaNaoEncontrada(id)

  call.respond(tarefa)
}
```

```java
public class TarefaNaoEncontrada extends RuntimeException {
  public TarefaNaoEncontrada(int id) { super("Tarefa " + id + " não encontrada"); }
}

@GET
@Path("/{id}")
public Tarefa buscar(@RestPath int id) {
  return repositorio.buscar(id).orElseThrow(() -> new TarefaNaoEncontrada(id));
}
```

A rota fica com o caminho feliz. O `?:` do Kotlin (seção 4.2) e o `orElseThrow` do `Optional` fazem o mesmo papel: "ou o valor, ou a exceção".

`BadRequestException` (pacote `io.ktor.server.plugins`) é uma exceção do Ktor que já corresponde a `400`. O Jakarta REST tem equivalentes, como `NotFoundException` e `BadRequestException` (pacote `jakarta.ws.rs`); lançá-las gera o status certo, mas sem corpo (conferido com `NotFoundException`: `404` vazio). Para o formato da Sprint 0, é preciso um tratador próprio.

6.6 `application/problem+json` (RFC 9457)

A RFC 9457 define um corpo padrão para erros, visto na Sprint 0:

```
HTTP/1.1 404 Not Found
Content-Type: application/problem+json

{
  "type": "https://tarefas.exemplo.br/erros/tarefa-nao-encontrada",
  "title": "Tarefa não encontrada",
  "status": 404,
  "detail": "Tarefa 99 não encontrada",
  "instance": "/tarefas/99"
}
```

Membro	Significado
`type`	URI que identifica a classe do erro; o cliente decide o que fazer por ele
`title`	resumo legível da classe do erro; não muda de uma ocorrência para outra
`status`	o mesmo código da resposta HTTP
`detail`	explicação desta ocorrência

Membro	Significado
`instance`	URI desta ocorrência (aqui, o caminho da requisição)

Quando `type` é omitido, vale `about:blank`, que significa "nenhuma semântica além do status"; nesse caso, a RFC recomenda que `title` seja a frase padrão do status (`Not Found` para `404`). Para erros próprios do domínio, use uma URI de `type` própria.

No Ktor, o tratador central é o plugin `StatusPages` (dependência `ktor-server-status-pages`):

```kotlin
@Serializable
data class Problema(
  val type: String = "about:blank",
  val title: String,
  val status: Int,
  val detail: String? = null,
  val instance: String? = null,
)

suspend fun ApplicationCall.responderProblema(problema: Problema) {
  respondText(
    Json.encodeToString(problema),
    ContentType.Application.ProblemJson,
    HttpStatusCode.fromValue(problema.status),
  )
}

fun Application.erros() {
  install(StatusPages) {
    exception<TarefaNaoEncontrada> { call, causa ->
      call.responderProblema(Problema(
        type = "https://tarefas.exemplo.br/erros/tarefa-nao-encontrada",
        title = "Tarefa não encontrada", status = 404,
        detail = causa.message, instance = call.request.path()))
    }
    exception<BadRequestException> { call, causa ->
      call.responderProblema(Problema(title = "Bad Request", status = 400,
        detail = causa.message, instance = call.request.path()))
    }
    exception<Throwable> { call, causa ->
      val status = defaultExceptionStatusCode(causa) ?: HttpStatusCode.InternalServerError
      if (status == HttpStatusCode.InternalServerError) {
        call.application.environment.log.error("erro não tratado", causa)
      }
      call.responderProblema(Problema(title = status.description, status = status.value,
        instance = call.request.path()))
    }
  }
}
```

Três decisões nesse código:

- `respondText` com `ContentType.Application.ProblemJson` garante o cabeçalho `application/problem+json`. Um `call.respond(problema)` passaria pelo `ContentNegotiation`, que escolheria `application/json`.
- `Json.encodeToString` usa as configurações padrão do kotlinx (seção 5.4): `detail` e `instance` nulos não aparecem no JSON, e `type` com o valor padrão `about:blank` também não, o que a RFC permite (ausente equivale a `about:blank`).
- O tratador de `Throwable` usa `defaultExceptionStatusCode`, função pública do Ktor que devolve o status que o próprio Ktor usaria para suas exceções (`415`, `413`...). Sem isso, o tratador genérico transforma esses casos em `500` (ver erro comum na seção 6.8).

A ordem dos `exception<...>` não importa: o `StatusPages` escolhe o tratador da classe mais próxima da exceção lançada. Conferido declarando `Throwable` antes de `RuntimeException` e de `TarefaNaoEncontrada`: cada exceção caiu no tratador mais específico.

No Quarkus, o equivalente é um método anotado com `@ServerExceptionMapper`, numa classe qualquer:

```
public record Problema(String type, String title, int status, String detail, String
        instance) {}


public class MapeadorDeErros {


  @ServerExceptionMapper
  public Response tarefaNaoEncontrada(TarefaNaoEncontrada causa, UriInfo uri) {
    return Response.status(404)
        .type("application/problem+json")
        .entity(new Problema("https://tarefas.exemplo.br/erros/tarefa-nao-encontrada",
            "Tarefa não encontrada", 404, causa.getMessage(), uri.getPath()))
        .build();
  }
}
```

O tipo do primeiro parâmetro define qual exceção o método trata. `UriInfo` é injetado pelo Quarkus e fornece o caminho. O MUSI usa a forma padrão do Jakarta REST, uma classe `@Provider` que implementa `ExceptionMapper<T>`; as duas funcionam.

Resultado conferido nos dois stacks:

```
$ curl -i localhost:8080/tarefas/99
HTTP/1.1 404 Not Found
Content-Type: application/problem+json

{"type":"https://tarefas.exemplo.br/erros/tarefa-nao-encontrada","title":"Tarefa não
encontrada","status":404,"detail":"Tarefa 99 não encontrada","instance":"/tarefas/99"}
```

📖 Ref. RFC 9457 — Problem Details for HTTP APIs: https://www.rfc-editor.org/rfc/rfc9457.html

📖 Ref. Ktor — Status pages: https://ktor.io/docs/server-status-pages.html

📖 Ref. Quarkus REST — Exception mapping: https://quarkus.io/guides/rest#exception-mapping

6.7 Validação: `400` ou `422`?

O capítulo 5 mostrou que o tipo sozinho não barra `{"titulo":" "}`. Validação é a regra além do tipo. Antes do código, a escolha do status:

- `400 Bad Request`: a requisição não pôde ser entendida (JSON malformado, tipo errado).
- `422 Unprocessable Content`: a requisição foi entendida, mas viola uma regra (título em branco). É o status usado nos exemplos da Sprint 0.

No Ktor, o plugin `RequestValidation` (dependência `ktor-server-request-validation`) valida o objeto logo depois do `receive`:

```
install(RequestValidation) {
  validate<NovaTarefa> { nova ->
    if (nova.titulo.isBlank()) ValidationResult.Invalid("titulo não pode ser vazio")
    else ValidationResult.Valid
  }
}

// no StatusPages:
exception<RequestValidationException> { call, causa ->
  call.responderProblema(Problema(title = "Dados inválidos", status = 422,
    detail = causa.reasons.joinToString("; "), instance = call.request.path()))
}
```

No Quarkus, a extensão `quarkus-hibernate-validator` usa as anotações do Jakarta Validation:

```
public record NovaTarefa(@NotBlank String titulo) {}

@POST
public Response criar(@Valid @NotNull NovaTarefa nova) { /* ... */ }
```

`@Valid` pede a validação das anotações de dentro de `NovaTarefa`; `@NotNull` barra o corpo vazio, que sem ela chegava como `null` e virava `NullPointerException` (seção 5.6).

Sem mais nada, o Quarkus responde `400` com um corpo próprio, em `application/json` (conferido):

```
{"title":"Constraint Violation","status":400,"violations":
        [{"field":"criar.arg0.titulo","message":"must not be blank"}]}
```

O `arg0` aparece porque o exemplo não compila com `-parameters` (seção 2.3). Para seguir o formato do projeto, um mapeador próprio de `ConstraintViolationException` substitui o padrão (conferido):

```
@ServerExceptionMapper
public Response validacao(ConstraintViolationException causa, UriInfo uri) {
  String detalhe = causa.getConstraintViolations().stream()
      .map(v -> v.getPropertyPath() + ": " + v.getMessage())
      .collect(Collectors.joining("; "));
  return Response.status(422)
      .type("application/problem+json")
      .entity(new Problema("https://tarefas.exemplo.br/erros/dados-invalidos",
          "Dados inválidos", 422, detalhe, uri.getPath()))
      .build();
}
```

Entrada	Ktor (com `RequestValidation` e `StatusPages`)	Quarkus (com Hibernate Validator e mapeador)
`{"titulo":"Ler"}`	`201`	`201`
`{"titulo":" "}`	`422`, problem+json	`422`, problem+json
`{}`	`400` (falha na conversão, capítulo 5)	`422` (`@NotBlank` pega o `null`)
corpo vazio	`400`	`422` (`@NotNull`)
`{"titulo":`	`400`	`400`

📖 Ref. Ktor — Request validation: https://ktor.io/docs/server-request-validation.html

📖 Ref. Quarkus — Validation with Hibernate Validator: https://quarkus.io/guides/validation

📖 Ref. RFC 9110 — 422 Unprocessable Content: https://www.rfc-editor.org/rfc/rfc9110.html#section-15.5.21

6.8 Erros comuns com respostas e erros

> Erro comum (Ktor): `exception<Throwable>` que devolve sempre `500`. O tratador genérico captura também as exceções que o Ktor transformaria em outro status. Com ele respondendo `500` fixo, um `POST` com `Content-Type: text/plain` passou de `415` para `500` (conferido). Use `defaultExceptionStatusCode(causa)` para preservar esses casos, como na seção 6.6.

> Erro comum (Ktor): `status(HttpStatusCode.NotFound) { ... }` no `StatusPages`. O tratador por status substitui toda resposta `404`, inclusive as que a rota enviou com corpo: no teste, `call.respondText("sumiu", status = NotFound)` saiu com o corpo do tratador. Se usar tratador por status, as rotas não devem responder `404` diretamente; lancem exceção.

> Erro comum (os dois): vazar detalhes internos no `500`. Colocar `causa.message` de uma exceção desconhecida em `detail` pode expor SQL, caminhos de arquivo ou nomes de classe. Para `500`, registre a causa no log e devolva só o genérico. No Quarkus, o tratador padrão já faz a separação: nos perfis dev e test, o corpo do `500` traz a pilha de chamadas; em prod, só um identificador do erro que aparece também no log (conferido).

> Erro comum (Quarkus): esquecer `@NotNull` no corpo. `@Valid` valida os campos de um objeto que existe. Com corpo vazio, o parâmetro chega `null`, `@Valid` não tem o que validar, e o método quebra com `500`.

> Erro comum (os dois): devolver `200` para criação e `200` com corpo vazio para remoção. Funciona, mas descumpre o contrato da Sprint 1: `POST` que cria devolve `201` com `Location`; remoção sem corpo devolve `204`.

---

7. Arquitetura: regra de dependência, portas e repositório

Até aqui, o código foi organizado por uma decisão tomada no passo 4 da aula: a rota fala com uma interface `RepositorioDeTarefas`, e a implementação em memória fica separada. Este capítulo explica o porquê dessa decisão, de onde ela vem e quando ela não vale o custo.

7.1 O ponto de partida: a rota que faz tudo

No passo 3, a lista morava dentro das rotas:

```
routing {
  val tarefas = mutableListOf(Tarefa(1, "Estudar Ktor"))
  var proximoId = 2
  post("/tarefas") {
    val nova = call.receive<NovaTarefa>()
    val criada = Tarefa(proximoId++, nova.titulo)
    tarefas.add(criada)
    call.respond(HttpStatusCode.Created, criada)
  }
}
```

Para um exemplo de 20 linhas, é o código mais simples possível. Os problemas aparecem quando o projeto cresce:

- Trocar a lista por um banco (21/09) exige reescrever cada handler.
- Testar a regra "o id é atribuído pelo servidor" exige subir HTTP.
- A mesma regra (gerar id, validar título) tende a ser copiada entre rotas.
- HTTP, regra de negócio e armazenamento mudam por motivos diferentes, e estão no mesmo bloco.

7.2 A regra de dependência

A Clean Architecture (Robert C. Martin) organiza o código em camadas concêntricas e impõe uma regra só: dependências de código-fonte apontam para dentro. O centro não conhece a borda.



Em termos de `import`:

Arquivo	Pode importar	Não pode importar
`Tarefa`, `RepositorioDeTarefas` (domínio)	biblioteca padrão	Ktor, Jakarta REST, JDBC, Koin
`RepositorioEmMemoria` (adaptador)	domínio	rotas
`Rotas.kt`, `RecursoDeTarefas` (adaptador)	domínio, framework HTTP	a implementação concreta do repositório

O ponto que parece contraditório: a rota depende do repositório em tempo de execução (ela chama `listar()`), mas o código da rota não depende da implementação. Ele depende da interface, que mora no domínio. A implementação também depende da interface (ela a implementa). As duas setas apontam para dentro. Isso é inversão de dependência.

Um ponto honesto sobre o exemplo: `@Serializable` em `Tarefa` (Ktor) e `@NotBlank` em `NovaTarefa` (Quarkus) são anotações de biblioteca dentro de tipos que chamamos de domínio. O MUSI resolve com tipos de transporte separados (`Dtos.kt`, `Dtos.java`) e mapeamento na borda. No exemplo da aula, a simplificação é aceitável; num projeto em que o formato público e o modelo interno evoluem separados, os tipos de transporte se pagam.

📖 Ref. Robert C. Martin — The Clean Architecture: https://blog.cleancoder.com/uncle-bob/2012/08/13/the-clean-architecture.html

7.3 Portas e adaptadores

Alistair Cockburn descreveu a mesma ideia como arquitetura hexagonal, ou portas e adaptadores:

- Porta é uma interface definida pela aplicação, nos termos dela.
- Adaptador é uma implementação que liga a porta a uma tecnologia.

Há dois lados:



À esquerda, adaptadores HTTP e testes acionam a aplicação. À direita, a aplicação aciona armazenamento através de uma porta que ela mesma define. Trocar um adaptador não muda a aplicação: é a promessa "na parte 2, trocamos memória por Postgres sem mexer nas rotas". O MUSI usa esse vocabulário nos pacotes (`adaptadores/web` , por exemplo).

📖 Ref. Alistair Cockburn — Hexagonal Architecture: https://alistair.cockburn.us/hexagonal-architecture/

7.4 O padrão Repositório

Martin Fowler define o repositório como um mediador entre o domínio e o mapeamento de dados, com uma interface parecida com uma coleção em memória. Três características práticas:

- A interface fala a língua do domínio: `buscar(id)` , `adicionar(nova)` , `remover(id)` . Não fala de tabelas, `ResultSet` ou SQL.
- Quem usa não sabe onde os dados estão. Memória, Postgres ou um serviço remoto são detalhes da implementação.
- Em Domain-Driven Design, há um repositório por agregado (a entidade raiz e o que vive junto com ela), não um por tabela.

A interface usada daqui para frente, nos dois stacks:

```
interface RepositorioDeTarefas {
  fun listar(): List<Tarefa>
  fun buscar(id: Int): Tarefa?
  fun adicionar(nova: NovaTarefa): Tarefa
  fun remover(id: Int): Boolean
}
```

```java
public interface RepositorioDeTarefas {
  List<Tarefa> listar();
  Optional<Tarefa> buscar(int id);
  Tarefa adicionar(NovaTarefa nova);
  boolean remover(int id);
}
```

"Não encontrado" é resultado esperado de `buscar`, não falha; por isso a interface devolve `Tarefa?` (Kotlin) ou `Optional<Tarefa>` (Java). Quem decide se isso vira `404` é a borda HTTP (seção 6.5).

Quando não usar: se o repositório só repassa cada chamada para uma biblioteca que já oferece a mesma interface, ele vira uma camada de cerimônia. O Panache (21/09), no modo repositório, já é um repositório pronto; envolvê-lo em outro raramente compensa. O critério é perguntar o que a interface esconde. Se a resposta for "nada", ela sobra.

📖 Ref. Martin Fowler — Repository: https://martinfowler.com/eaaCatalog/repository.html

📖 Ref. Microsoft Learn — Padrão de repositório (PT): https://learn.microsoft.com/pt-br/dotnet/architecture/microservices/microservice-ddd-cqrs-patterns/infrastructure-persistence-layer-design

7.5 A interface como fronteira

A interface só protege se o que atravessa a fronteira for do domínio:

Assinatura	Situação	Por quê
`fun buscar(id: Int): Tarefa?`	adequada	só tipos do domínio
`fun buscar(call: ApplicationCall)`	evitar	amarra o repositório ao Ktor
`fun executar(sql: String): ResultSet`	evitar	expõe a tecnologia; trocar de banco muda os chamadores
`Tarefa buscar(int id) throws SQLException`	evitar	a exceção vaza o detalhe; o adaptador deve traduzi-la
`Response remover(int id)`	evitar	tipo HTTP no domínio; a decisão de status é da borda

Um teste rápido: dá para imaginar uma segunda implementação razoável (memória e Postgres) sem mudar a assinatura? Se não, a interface está vazando.

7.6 Testabilidade

O ganho mais imediato da interface aparece nos testes. Um repositório falso, escrito à mão, substitui o real:

```
class RepositorioFalso(private val tarefas: List<Tarefa>) : RepositorioDeTarefas {
  override fun listar() = tarefas
  override fun buscar(id: Int) = tarefas.find { it.id == id }
  override fun adicionar(nova: NovaTarefa) = Tarefa(99, nova.titulo)
  override fun remover(id: Int) = false
}
```

No Ktor, se as rotas recebem o repositório por parâmetro, o teste monta a aplicação com o falso, sem Koin:

```
fun Route.rotasDeTarefas(repositorio: RepositorioDeTarefas) {
  route("/tarefas") { /* get, post, delete... */ }
}


@Test
fun `lista o que o repositório devolve`() = testApplication {
  application {
    install(ContentNegotiation) { json() }
    routing { rotasDeTarefas(RepositorioFalso(listOf(Tarefa(7, "Falsa")))) }
  }
  assertEquals("""[{"id":7,"titulo":"Falsa","feita":false}]""",
        client.get("/tarefas").bodyAsText())
}
```

No Quarkus, o recurso é uma classe Java comum. Com o campo `repositorio` visível no pacote (sem `private`, como recomenda a documentação do CDI), um teste JUnit puro, sem subir o Quarkus, roda em milissegundos:

```java
class RecursoDeTarefasUnitarioTest {

  @Test
  void buscarInexistenteLancaExcecao() {
    var recurso = new RecursoDeTarefas();
    recurso.repositorio = new RepositorioFalso();
    assertThrows(TarefaNaoEncontrada.class, () -> recurso.buscar(1));
  }
}
```

Os dois testes foram executados. Vocabulário útil, de Gerard Meszaros via Fowler:

Dublê	O que faz
dummy	só preenche um parâmetro; nunca é usado
stub	devolve respostas prontas
fake	implementação funcional simplificada (o `RepositorioFalso` acima)
spy	stub que registra como foi chamado
mock	é programado com expectativas e falha se elas não se cumprirem

📖 Ref. Martin Fowler — Test Double: https://martinfowler.com/bliki/TestDouble.html

7.7 Estrutura de pacotes sugerida

```
br.ufrn.<projeto>/
├── dominio/            Tarefa, RepositorioDeTarefas, TarefaNaoEncontrada
├── aplicacao/          casos de uso, quando houver regra além do CRUD
├── adaptadores/
│   ├── web/            Rotas.kt / RecursoDeTarefas.java, Problema, mapeadores de erro
│   └── persistencia/   RepositorioEmMemoria (hoje), RepositorioPostgres (21/09)
└── Aplicacao.kt        montagem: plugins, Koin (no Quarkus, não há arquivo equivalente)
```

A pasta `aplicacao/` só aparece quando houver regra que não pertence nem à rota nem ao repositório (por exemplo, "não é possível concluir tarefa de outro usuário"). Um `ServicoDeTarefas` que só repassa chamadas ao repositório é uma camada anêmica: mais um arquivo sem decisão nenhuma.

A rubrica da Sprint 1 pede um teste de arquitetura que falhe quando o domínio importar framework ou infraestrutura. Bibliotecas como ArchUnit (Java) e Konsist (Kotlin) expressam regras de dependência entre pacotes como testes comuns; a estrutura acima é o que torna essas regras simples de escrever.

📖 Ref. ArchUnit: https://www.archunit.org/

📖 Ref. Konsist: https://docs.konsist.lemonappdev.com/

7.8 Erros comuns de arquitetura

> Erro comum: a interface do repositório com cara de banco. `salvarLinha`, `executarQuery`, parâmetros `Map<String, Any>`: a implementação vazou para a porta. Nomeie operações pelo que o domínio precisa.

> Erro comum: um repositório genérico para tudo. `Repositorio<T>` com `findAll`, `save` e `delete` para cada entidade tira do domínio a chance de dizer quais operações fazem sentido (uma tarefa pode ser removida; um registro de auditoria, não).

> Erro comum: domínio importando framework por conveniência. Um `import io.ktor...` ou `import jakarta.ws.rs...` em `dominio/` é o sinal. Quando o teste de arquitetura existir, é exatamente isso que ele pega.

> Erro comum: criar camadas antes de ter o que pôr nelas. Interface para cada classe, serviço que só repassa, DTO idêntico à entidade. Cada camada precisa esconder uma decisão que pode mudar.

---

8. Injeção de dependência: Koin × CDI

A rota precisa de um `RepositorioDeTarefas`. Alguém tem de criar o `RepositorioEmMemoria` e entregá-lo. Injeção de dependência é a técnica de a classe receber o que precisa em vez de construir; um contêiner de injeção (Koin, CDI) automatiza essa entrega.

8.1 Quem usa não constrói

```
// sem injeção: a rota escolhe a implementação
fun Application.rotas() {
  val repositorio = RepositorioEmMemoria()   // trocar por Postgres = editar a rota
  // ...
}
```

A linha acima desfaz o capítulo 7: a rota voltou a conhecer a implementação. As formas de receber a dependência, segundo o artigo clássico de Fowler:

Forma	Exemplo	Observação
construtor	`class Servico(private val repo: RepositorioDeTarefas)`	dependência explícita e obrigatória; preferida
propriedade ou campo	`@Inject RepositorioDeTarefas repositorio;`	comum em CDI; o objeto existe antes de receber a dependência
localizador de serviço	`val repo = get<RepositorioDeTarefas>()`	a classe pede ao contêiner; depende dele

O `by inject()` do Koin é, tecnicamente, um localizador de serviço preguiçoso: a função pede a dependência ao contêiner no primeiro uso.

📖 Ref. Martin Fowler — Inversion of Control Containers and the Dependency Injection pattern: https://martinfowler.com/articles/injection.html

8.2 Injeção sem biblioteca

Antes de escolher contêiner, vale lembrar que injeção não exige um. No Ktor, passar a dependência por parâmetro resolve (é o que a documentação do Ktor chama de dependências por parâmetro dos módulos):

```
fun main() {
  val repositorio: RepositorioDeTarefas = RepositorioEmMemoria()
  embeddedServer(CIO, port = 8080) {
    install(ContentNegotiation) { json() }
    routing { rotasDeTarefas(repositorio) }
  }.start(wait = true)
}
```

Todas as dependências aparecem no `main`, o compilador verifica cada uma, e o teste da seção 7.6 usa a mesma função com um falso. Para aplicações pequenas, é suficiente. Um contêiner começa a compensar quando o grafo cresce (dezenas de objetos, dependências entre eles, escopos diferentes).

O Ktor também tem um plugin de injeção próprio (`ktor-server-di`). O curso usa Koin, como o MUSI.

📖 Ref. Ktor — Modules (dependências entre módulos): https://ktor.io/docs/server-modules.html

📖 Ref. Ktor — Dependency injection: https://ktor.io/docs/server-dependency-injection.html

8.3 Koin

O Koin declara o grafo em código, num `module`:

```
val moduloDeTarefas = module {
  single<RepositorioDeTarefas> { RepositorioEmMemoria() }   // forma do exemplo da aula
}

fun Application.modulo() {
  install(Koin) { modules(moduloDeTarefas) }
  // ...
}
```

Os tipos de definição:

Definição	Instância	Paralelo
`single { }`	uma para a aplicação inteira, criada no primeiro pedido	`@ApplicationScoped`, `@Singleton` do Spring
`factory { }`	uma nova a cada pedido	`@Dependent`, `prototype` do Spring
`scoped { }`	uma por escopo (por exemplo, por requisição)	`@RequestScoped`

Conferido com `koinApplication`: dois `get<T>()` de um `single` devolvem o mesmo objeto; de um `factory`, objetos diferentes.

Quando a classe recebe dependências pelo construtor, o Koin monta a chamada com `get()`, ou com referência ao construtor:

```
class ServicoDeTarefas(private val repositorio: RepositorioDeTarefas)

val moduloDeTarefas = module {
  singleOf(::RepositorioEmMemoria) bind RepositorioDeTarefas::class
  single { ServicoDeTarefas(get()) }      // get() resolve o RepositorioDeTarefas
  // ou: singleOf(::ServicoDeTarefas)
}
```

`singleOf(::RepositorioEmMemoria)` registra pelo tipo concreto; `bind` acrescenta a interface. Sem o `bind`, quem pede `RepositorioDeTarefas` não encontra nada.

Para obter a dependência:

```
fun Application.rotas() {
  val repositorio by inject<RepositorioDeTarefas>()   // preguiçoso: resolve no primeiro uso
  val outro = get<RepositorioDeTarefas>()              // imediato: resolve nesta linha
  // ...
}
```

A diferença entre os dois muda quando o erro aparece. Com um módulo sem a definição (conferido):

- `by inject`, usado só dentro do handler: a aplicação sobe; a primeira requisição falha com `NoDefinitionFoundException` e o cliente recebe `500`.
- a dependência usada durante a montagem (por exemplo, passada para `rotasDeTarefas`): a aplicação nem sobe.

Falhar na subida é melhor que falhar na primeira requisição em produção. Melhor ainda é falhar num teste. O `koin-test` oferece `verify()`, que confere se cada parâmetro de construtor tem definição:

```
@Test
fun `modulo de tarefas esta completo`() {
  moduloDeTarefas.verify()
}
```

Com `singleOf(::ServicoDeTarefas)` e sem o repositório, o teste falha com `MissingKoinDefinitionException: Missing definition for '[field:'repositorio' – type:'...RepositorioDeTarefas']' in definition '[Singleton: 'ServicoDeTarefas']'` (conferido). O `verify()` analisa construtores; dependências obtidas com `get()` dentro de lambdas não entram na análise.

Escopo por requisição, com o `koin-ktor`:

```
class ContextoDaRequisicao

val modulo = module {
  requestScope {
    scopedOf(::ContextoDaRequisicao)
  }
}

get("/x") {
  val contexto = call.scope.get<ContextoDaRequisicao>()   // o mesmo objeto durante esta
        requisição
}
```

Conferido: dentro de uma requisição, dois `call.scope.get()` devolvem o mesmo objeto; na requisição seguinte, outro.

O Koin 4.2 também oferece anotações e um plugin de compilador que verifica o grafo no build. O exemplo da aula e o MUSI usam a DSL clássica mostrada aqui.

📖 Ref. Koin — Definitions: https://insert-koin.io/docs/reference/koin-core/definitions

📖 Ref. Koin — Koin for Ktor: https://insert-koin.io/docs/reference/koin-ktor/ktor

📖 Ref. Koin — Request scopes no Ktor: https://insert-koin.io/docs/reference/koin-ktor/ktor-scopes

📖 Ref. Koin — Verify: https://insert-koin.io/docs/reference/koin-test/verify

8.4 CDI no Quarkus

CDI (Contexts and Dependency Injection) é a especificação de injeção do Jakarta EE. O Quarkus a implementa com o ArC, que resolve o grafo durante o build.

Uma classe vira bean quando tem uma anotação de escopo; quem precisa dela usa `@Inject`:

```
@ApplicationScoped
public class RepositorioEmMemoria implements RepositorioDeTarefas { /* ... */ }

@Path("/tarefas")
public class RecursoDeTarefas {
  @Inject
  RepositorioDeTarefas repositorio;
}
```

Os escopos mais usados:

Escopo	Instância	Criação
`@ApplicationScoped`	uma para a aplicação	preguiçosa: no primeiro método chamado; injeta-se um proxy
`@Singleton`	uma para a aplicação	quando é injetada; sem proxy
`@RequestScoped`	uma por requisição HTTP	na primeira chamada dentro da requisição
`@Dependent`	uma por ponto de injeção	quando é injetada

A documentação do Quarkus recomenda `@ApplicationScoped` como padrão. O proxy permite criação preguiçosa, injetar um bean de escopo menor (`@RequestScoped`) num maior e trocar o bean por um mock em testes.

A regra de resolução é estrita: para cada ponto de injeção, deve existir exatamente um bean compatível. Senão, o build falha (conferido):

```
# RepositorioEmMemoria sem @ApplicationScoped (nenhum bean)
UnsatisfiedResolutionException: Unsatisfied dependency for type
br.ufrn.exemplo.tarefas.RepositorioDeTarefas and qualifiers [@Default]

# duas classes implementando RepositorioDeTarefas, ambas com @ApplicationScoped
AmbiguousResolutionException: Ambiguous dependencies for type
br.ufrn.exemplo.tarefas.RepositorioDeTarefas and qualifiers [@Default]
  - available beans:
    - CLASS bean [... target=br.ufrn.exemplo.tarefas.RepositorioEmMemoria]
    - CLASS bean [... target=br.ufrn.exemplo.tarefas.RepositorioEmArquivo]
```

Para ter duas implementações, é preciso dizer qual usar: um qualificador (`@Named("memoria")` na classe e no ponto de injeção), `@Alternative` com `@Priority`, ou o `@DefaultBean` do Quarkus na implementação que vale quando não houver outra.

Detalhes específicos do Quarkus que diferem do CDI tradicional:

- Injeção por construtor dispensa construtor sem argumentos e, com um único construtor, dispensa até o `@Inject`.
- A documentação recomenda campos injetados visíveis no pacote, não `private`: para membros privados, o Quarkus precisa de reflexão.
- Beans que nenhum ponto de injeção usa são removidos no build.

Como fornecer um objeto que não é uma classe sua (um cliente de biblioteca configurado, por exemplo)? Com um método produtor:

```
// @Produces aqui é jakarta.enterprise.inject.Produces, não o jakarta.ws.rs.Produces do
        capítulo 2.
// RepositorioEmMemoria, nesse caso, fica sem anotação de escopo; senão haveria dois beans.
@ApplicationScoped
public class Configuracao {
  @Produces
  @ApplicationScoped
  RepositorioDeTarefas repositorio() {
    return new RepositorioEmMemoria();   // aqui pode haver lógica de montagem
  }
}
```

É o equivalente ao `single { ... }` do Koin e ao `@Bean` do Spring.

📖 Ref. Quarkus — Introduction to CDI: https://quarkus.io/guides/cdi

📖 Ref. Quarkus — CDI reference: https://quarkus.io/guides/cdi-reference

📖 Ref. Jakarta CDI: https://jakarta.ee/specifications/cdi/

8.5 Paralelo com o Spring

Parte da turma já viu Spring. A tabela serve de dicionário:

Spring	CDI (Quarkus)	Koin
`@Component`, `@Service`, `@Repository`	`@ApplicationScoped`	`single { }`
`@Autowired`	`@Inject`	`by inject()` / `get()` / parâmetro de construtor
`@Bean` num `@Configuration`	`@Produces`	`single { }` com lógica
`@Scope("prototype")`	`@Dependent`	`factory { }`
`@RequestScope`	`@RequestScoped`	`requestScope { scoped { } }`
`@Qualifier("x")`	`@Named("x")` ou qualificador próprio	`named("x")`
`@Primary`	`@Alternative` + `@Priority`	—
erro de resolução na subida	erro de resolução no build	erro na resolução (ou em `verify()`)

📖 Ref. Spring — Dependency Injection: https://docs.spring.io/spring-framework/reference/core/beans/dependencies/factory-collaborators.html

8.6 Koin × CDI

	Koin	CDI (Quarkus)
Onde se declara	módulos em código Kotlin	anotações nas classes
Visão do grafo	um arquivo lista tudo	espalhada; a Dev UI do Quarkus lista os beans
Quando erros aparecem	em execução (ou em `verify()` num teste)	no build
Criação padrão	`single` no primeiro pedido	`@ApplicationScoped` no primeiro método chamado
Escopos	`single`, `factory`, `scoped`	`@ApplicationScoped`, `@Singleton`, `@RequestScoped`, `@Dependent`
Custo de entrada	ler o módulo	conhecer escopos, proxies e regras de resolução

8.7 Erros comuns com injeção

> Erro comum (Quarkus): esquecer a anotação de escopo na implementação. Sem `@ApplicationScoped` (ou outro escopo), a classe não é bean, e o build falha com `UnsatisfiedResolutionException`. A mensagem cita o tipo pedido; procure a implementação dele.

> Erro comum (Quarkus): efeito colateral no construtor de um bean `@ApplicationScoped`. O proxy é uma subclasse gerada, e o construtor roda também para ele. No teste, um `println` no construtor de `RepositorioEmMemoria` apareceu duas vezes: uma para `RepositorioEmMemoria_ClientProxy`, outra para a instância real. Inicialização vai num método `@PostConstruct`.

> Erro comum (Quarkus): ler ou escrever campos de um bean `@ApplicationScoped` injetado. O que foi injetado é o proxy, que só repassa chamadas de método. Acesse sempre por métodos.

> Erro comum (Koin): `singleOf(::Implementacao)` sem `bind`. O bean existe, mas registrado só pelo tipo concreto. Quem pede a interface recebe `NoDefinitionFoundException`.

> Erro comum (os dois): `new` escondido dentro de uma classe gerenciada. Um `private val repo = RepositorioEmMemoria()` dentro de um serviço injetado anula a injeção para essa dependência: o teste não consegue trocá-la.

---

9. Concorrência: não travar a thread

Um servidor HTTP atende muitas requisições ao mesmo tempo. A maior parte do tempo de uma requisição típica é espera: pelo banco, por outro serviço, pelo disco. Este capítulo trata de como cada stack lida com essa espera, com medições feitas nos exemplos.

As medições usaram endpoints de teste que esperam 1 segundo, chamados por 300 clientes simultâneos (`curl` em paralelo), numa máquina de 8 núcleos, com a aplicação empacotada (não em modo dev). Os números exatos variam de máquina para máquina; as proporções são o que importa.

9.1 O modelo clássico: uma thread por requisição

No modelo Servlet tradicional, cada requisição ocupa uma thread de um pool do início ao fim. Enquanto a requisição espera o banco, a thread fica parada, sem fazer nada, mas ocupada.

```
pool com 4 threads, 6 requisições que esperam o banco

thread 1  [req A ....espera....]
thread 2  [req B ....espera....]
thread 3  [req C ....espera....]
thread 4  [req D ....espera....]
          req E e req F aguardam na fila, embora a CPU esteja ociosa
```

Aumentar o pool ajuda até certo ponto: threads de plataforma (as do sistema operacional) custam memória e troca de contexto. Os dois stacks da aula oferecem alternativas, e as threads virtuais do Java 21 oferecem uma terceira.

📖 Ref. Jakarta Servlet: https://jakarta.ee/specifications/servlet/

9.2 Corrotinas e `suspend`

Uma função `suspend` pode pausar num ponto de suspensão (a chamada a outra função `suspend`) e retomar depois. Enquanto está pausada, a thread fica livre para outra corrotina. O handler do Ktor já é `suspend` (seção 4.6: `suspend RoutingContext.() -> Unit`).

```
get("/espera/suspensa") {
  delay(1000)                  // suspende: a thread atende outras requisições
  call.respondText("pronto")
}

get("/espera/bloqueante") {
  Thread.sleep(1000)           // bloqueia: a thread fica presa por 1 segundo
  call.respondText("pronto")
}
```

`delay` é uma função `suspend`; `Thread.sleep` é uma chamada bloqueante comum. As duas esperam 1 segundo, mas o efeito no servidor é muito diferente (Ktor 3.5.2 com CIO):

Endpoint	64 simultâneas	65 simultâneas	300 simultâneas
`Thread.sleep(1000)`	1,17 s	2,06 s	5,16 s

Endpoint	64 simultâneas	65 simultâneas	300 simultâneas
`delay(1000)`	1,14 s	1,15 s	1,60 s

Com espera bloqueante, até 64 requisições foram atendidas em paralelo; a 65ª já esperou uma rodada inteira, e 300 levaram cerca de cinco rodadas. Com `delay`, 300 requisições terminaram praticamente juntas.

O efeito não fica restrito ao endpoint lento. Com as 300 requisições bloqueantes em curso, um `GET /tarefas` comum levou 2,8 s. Com as 300 suspensas em curso, levou 2 ms. Código bloqueante num handler prejudica todas as rotas.

O ponto que costuma confundir: marcar uma função com `suspend` não torna o que está dentro dela não bloqueante.

```
suspend fun buscarNoBanco(): List<Tarefa> {
  return jdbc.executeQuery("select ...")   // JDBC é bloqueante; o suspend não muda isso
}
```

Uma chamada JDBC continua segurando a thread, com ou sem `suspend` na assinatura. A suspensão só acontece de verdade quando, lá no fundo, há uma função que suspende (um driver assíncrono, um cliente HTTP do Ktor, um `delay`).

📖 Ref. Kotlin — Coroutines basics: https://kotlinlang.org/docs/coroutines-basics.html

9.3 Dispatchers e `withContext`

O dispatcher decide em que threads uma corrotina roda:

Dispatcher	Para quê	Tamanho padrão
`Dispatchers.Default`	trabalho de CPU	número de núcleos (no mínimo 2)
`Dispatchers.IO`	chamadas bloqueantes (arquivo, JDBC)	64 ou o número de núcleos, o que for maior
`Dispatchers.Main`	thread de interface (Android, desktop)	não se usa em servidor
`Dispatchers.Unconfined`	casos especiais; roda na thread de quem chamou até a primeira suspensão	—

Quando não há alternativa suspensa (JDBC, por exemplo), a recomendação é isolar a chamada bloqueante em `Dispatchers.IO`:

```
get("/espera/io") {
  val resultado = withContext(Dispatchers.IO) {
    Thread.sleep(1000)          // bloqueia uma thread do pool de IO, não a do handler
    "pronto"
  }
  call.respondText(resultado)
}
```

Medido nas mesmas condições, esse endpoint teve exatamente os números da versão bloqueante: 1,15 s para 64, 2,06 s para 65, 5,16 s para 300. O limite observado, 64, é o limite padrão de `Dispatchers.IO` nesta máquina; a documentação registra que `Default` e `IO` compartilham threads. A lição: `withContext(Dispatchers.IO)` é a forma correta de chamar código bloqueante a partir de corrotinas (mantém as threads de CPU livres), mas não multiplica a capacidade de espera. Isso só vem de E/S que suspende de verdade ou de um pool maior dedicado, como `Dispatchers.IO.limitedParallelism(100)`.

Na parte 2 (21/09), o Exposed acessa o banco via JDBC. É exatamente o caso desta seção.

📖 Ref. Kotlin — Coroutine context and dispatchers: https://kotlinlang.org/docs/coroutine-context-and-dispatchers.html

📖 Ref. kotlinx.coroutines — `Dispatchers.IO`: https://kotlinlang.org/api/kotlinx.coroutines/kotlinx-coroutines-core/kotlinx.coroutines/-dispatchers/-i-o.html

9.4 Quarkus: event loop, threads de trabalho e threads virtuais

O Quarkus REST roda sobre o Vert.x, com dois tipos de thread:

- Event loop (threads de E/S): poucas, dimensionadas a partir do número de núcleos. Leem e escrevem bytes nas conexões e executam código que não bloqueia.
- Threads de trabalho (worker threads): um pool maior, para código que bloqueia. O tamanho máximo padrão é o maior entre 8 × núcleos e 200.

Qual das duas executa um método é decidido pela assinatura (conferido pelo nome da thread):

Retorno do método	Thread padrão	Nome observado
objeto comum (`List<Tarefa>` , `String` , `Response` , `void`)	trabalho	`executor-thread-1`
`Uni` , `Multi` , `CompletionStage` , `Publisher`	event loop	`vert.x-eventloop-thread-1`
função Kotlin `suspend`	event loop	—

`@Blocking` e `@NonBlocking` sobrescrevem a escolha; métodos com `@Transactional` são tratados como bloqueantes.

Isso corrige uma leitura comum (e a versão anterior deste guia): o código imperativo do exemplo, que devolve `List<Tarefa>`, não roda no event loop. Ele roda numa thread de trabalho, no modelo de uma thread por requisição, com um pool grande. O event loop entra em cena quando o método devolve um tipo reativo.

As medições com 300 requisições simultâneas de 1 segundo:

Endpoint	Thread	300 simultâneas
`String` com `Thread.sleep(1000)`	trabalho	2,24 s
`Uni` com espera não bloqueante (`onItem().delayIt()`)	event loop	1,60 s
`String` com `Thread.sleep(1000)` e `@RunOnVirtualThread`	virtual	1,60 s
`Uni` com `Thread.sleep(1000)` dentro	event loop	38,39 s

Leitura da tabela:

- O bloqueante em thread de trabalho escala até o tamanho do pool (200 aqui); por isso 2,24 s, e não 1 s.
- O `Uni` com espera real não bloqueante e a thread virtual atenderam as 300 juntas.
- O último caso é o erro grave: bloquear dentro de um método reativo prende uma thread do event loop, que são poucas. As 300 requisições levaram 38 segundos, e um `GET /tarefas` comum, feito durante o teste, levou 37 segundos. Com as bloqueantes em thread de trabalho, o mesmo `GET` levou 2 ms.

O Vert.x avisa no log quando uma thread do event loop fica bloqueada além do limite, que é de 2 segundos por padrão:

```
WARN [io.ver.cor.imp.BlockedThreadChecker] Thread Thread[vert.x-eventloop-thread-4,5,main]
has been blocked for 3196 ms, time limit is 2000 ms
```

Com esperas de 1 segundo, abaixo do limite, nenhum aviso apareceu, embora o servidor estivesse praticamente parado. A ausência de aviso não prova que está tudo bem.

Threads virtuais (Java 21+) são gerenciadas pela JVM: quando uma delas bloqueia em E/S, a JVM a desmonta da thread de plataforma, que fica livre. O resultado é escrever código bloqueante comum com capacidade de espera parecida com a do código assíncrono. Cuidados apontados pela documentação do Quarkus: não ajudam em trabalho de CPU; algumas situações prendem a thread de plataforma (pinning), e até o Java 23 isso incluía blocos `synchronized`, o que deixou de acontecer no Java 24 (JEP 491). O exemplo usa Java 25.

📖 Ref. Quarkus REST — Execution model, blocking, non-blocking: https://quarkus.io/guides/rest#execution-model-blocking-non-blocking

📖 Ref. Quarkus — Reactive architecture: https://quarkus.io/guides/quarkus-reactive-architecture

📖 Ref. Quarkus — Virtual thread support: https://quarkus.io/guides/virtual-threads

📖 Ref. JEP 444 — Virtual Threads: https://openjdk.org/jeps/444

9.5 Estado compartilhado: o repositório em memória não é seguro

Se várias requisições rodam ao mesmo tempo, código que altera dados compartilhados precisa de cuidado. O `RepositorioEmMemoria` dos dois exemplos usa uma lista comum e `proximoId++`:

```
private val tarefas = mutableListOf<Tarefa>()
private var proximoId = 3
override fun adicionar(nova: NovaTarefa) = Tarefa(proximoId++, nova.titulo).also {
        tarefas.add(it) }
```

Teste feito: 2.000 `POST` com 200 clientes simultâneos. Todas as respostas foram `201`, mas:

Stack	Tarefas esperadas	Tarefas na lista	Ids distintos
Ktor	2.002	1.896	1.895
Quarkus	2.002	1.872	1.869

Tarefas sumiram e ids se repetiram, sem nenhuma exceção no log. `proximoId++` são três operações (ler, somar, escrever), e `ArrayList.add` não é seguro para acesso concorrente. Duas threads intercaladas perdem uma escrita.

Uma correção simples, conferida no Ktor (2.002 tarefas, 2.002 ids distintos):

```
class RepositorioEmMemoria : RepositorioDeTarefas {
  private val tarefas = mutableListOf<Tarefa>()
  private var proximoId = 1
  private val trava = Any()

  override fun listar() = synchronized(trava) { tarefas.toList() }
  override fun adicionar(nova: NovaTarefa) = synchronized(trava) {
    Tarefa(proximoId++, nova.titulo).also { tarefas.add(it) }
  }
  // buscar e remover também dentro de synchronized(trava)
}
```

Em Java, a mesma ideia com métodos `synchronized`. Dentro de funções `suspend`, use `Mutex` (kotlinx.coroutines) em vez de `synchronized`: não se pode suspender segurando um monitor.

Na parte 2, o banco assume esse controle (transações e sequências geram ids sem colisão). O repositório em memória continua útil para testes de um cliente só.

9.6 Quando usar o quê

Situação	Ktor	Quarkus
CRUD com JDBC/ORM bloqueante	`withContext(Dispatchers.IO)` em volta da chamada	método comum (thread de trabalho), o padrão
Muitas chamadas lentas a outros serviços	cliente HTTP suspenso (Ktor client)	`Uni` com cliente reativo, ou `@RunOnVirtualThread`
Trabalho pesado de CPU	`Dispatchers.Default`	thread de trabalho
Código que você não controla e bloqueia	nunca direto no handler sem `withContext`	nunca em método que devolve `Uni`

Para o projeto da Sprint 1, com CRUD e banco relacional: no Ktor, isole o acesso ao banco em `Dispatchers.IO`; no Quarkus, mantenha métodos imperativos comuns e não misture tipos reativos com chamadas bloqueantes.

9.7 Erros comuns de concorrência

> Erro comum (Ktor): `Thread.sleep` ou JDBC direto no handler. Compila e funciona com um usuário. Sob carga, cada requisição segura uma thread do pool compartilhado, e todas as rotas ficam lentas (seção 9.2).

> Erro comum (Ktor): `runBlocking` dentro de um handler. `runBlocking` bloqueia a thread atual até o bloco terminar: é o oposto do que o handler `suspend` oferece. Ele serve para `main` e testes, não para código que já roda em corrotina.

> Erro comum (Quarkus): devolver `Uni` e bloquear dentro. O método vai para o event loop, e cada bloqueio para o servidor inteiro (seção 9.4). Se o código bloqueia, devolva o tipo comum ou anote `@Blocking`.

> Erro comum (os dois): achar que `suspend` ou `Uni` tornam o código assíncrono. A assinatura só diz onde o código vai rodar. O que está dentro precisa suspender ou ser assíncrono de verdade.

> Erro comum (os dois): estado mutável compartilhado sem sincronização. Beans `@ApplicationScoped`, `single` do Koin e recursos do Quarkus (uma instância para todas as requisições, conferido) são compartilhados entre threads. Lista, contador ou mapa dentro deles precisa de sincronização (seção 9.5).

---

10. Rodar, testar e comparar

10.1 Rodar

```
cd exemplos/ktor-tarefas    && ./gradlew run     # :8080
cd exemplos/quarkus-tarefas && mvn quarkus:dev   # :8081
```

Para rodar como em produção, empacotado:

```
# Ktor: plugin application do Gradle gera um script de execução
./gradlew installDist
build/install/ktor-tarefas/bin/ktor-tarefas

# Quarkus: jar executável em target/quarkus-app
mvn package
java -jar target/quarkus-app/quarkus-run.jar
```

Os dois foram executados assim para as medições do capítulo 9. O jar do Quarkus sobe com o perfil `prod` e escuta em `0.0.0.0` (seção 3.5).

10.2 Testar à mão

Uma sequência que percorre o contrato inteiro. Com o código dos capítulos 6 e 7 (rotas de busca e remoção, erros em problem+json), as respostas esperadas são:

```
curl -i localhost:8080/tarefas                                    # 200, lista
curl -i -X POST localhost:8080/tarefas \
  -H 'Content-Type: application/json' -d '{"titulo":"Ler"}'       # 201, Location
curl -i localhost:8080/tarefas/3                                  # 200, a tarefa
curl -i -X DELETE localhost:8080/tarefas/3                        # 204
curl -i -X DELETE localhost:8080/tarefas/3                        # 404, problem+json
curl -i -X POST localhost:8080/tarefas \
  -H 'Content-Type: application/json' -d '{"titulo":"  "}'        # 422, problem+json
curl -i -X POST localhost:8080/tarefas -d '{"titulo":"Ler"}'      # 415 (sem Content-Type)
```

Nas IDEs da JetBrains, o mesmo roteiro pode ficar num arquivo `.http` versionado no projeto, executado requisição por requisição:

```
### Listar
GET http://localhost:8080/tarefas

### Criar
POST http://localhost:8080/tarefas
Content-Type: application/json

{"titulo": "Ler o guia"}
```

📖 Ref. IntelliJ IDEA — HTTP Client: https://www.jetbrains.com/help/idea/http-client-in-product-code-editor.html

10.3 Testes automatizados no Ktor

O `ktor-server-test-host` sobe a aplicação em memória, sem abrir porta, e oferece um cliente HTTP ligado a ela:

```kotlin
class TarefasTest {

  @Test
  fun `lista as tarefas iniciais`() = testApplication {
    application { modulo() }

    val resposta = client.get("/tarefas")

    assertEquals(HttpStatusCode.OK, resposta.status)
    assertContains(resposta.bodyAsText(), "Estudar Ktor")
  }

  @Test
  fun `cria tarefa e devolve 201 com Location`() = testApplication {
    application { modulo() }

    val resposta = client.post("/tarefas") {
      contentType(ContentType.Application.Json)
      setBody("""{"titulo":"Ler o guia"}""")
    }

    assertEquals(HttpStatusCode.Created, resposta.status)
    assertEquals("/tarefas/3", resposta.headers[HttpHeaders.Location])
  }
}
```

- Cada `testApplication` cria uma aplicação nova, com repositório novo: os testes não compartilham estado.
- `application { modulo() }` usa a mesma montagem do `main`, com Koin e plugins. Conferido: vários testes seguidos, cada um instalando o Koin, rodam sem conflito.
- Nomes de teste entre crases (`` `cria tarefa e devolve 201` ``) são permitidos em Kotlin na JVM e leem como frases.
- Para trocar dependências, a forma mais simples é a da seção 7.6: montar as rotas com um falso. O Koin também oferece `KoinIsolated` para contextos separados por teste.

📖 Ref. Ktor — Testing: https://ktor.io/docs/server-testing.html

📖 Ref. Koin — Testing Ktor with Koin: https://insert-koin.io/docs/reference/koin-ktor/ktor-testing

10.4 Testes automatizados no Quarkus

`@QuarkusTest` sobe a aplicação numa porta de teste, e o RestAssured faz as requisições:

```
@QuarkusTest
class RecursoDeTarefasTest {

  @Test
  void listaAsTarefasIniciais() {
    given()
      .when().get("/tarefas")
      .then()
        .statusCode(200)
        .body("titulo", hasItem("Estudar Quarkus"));
  }

  @Test
  void criaTarefaEDevolve201ComLocation() {
    given()
        .contentType("application/json")
        .body("{\"titulo\":\"Ler o guia\"}")
      .when().post("/tarefas")
      .then()
        .statusCode(201)
        .header("Location", endsWith("/tarefas/3"));
  }
}
```

- `body("titulo", hasItem(...))` usa uma expressão sobre o JSON: `titulo` aplicado a uma lista devolve a lista de títulos.
- O Quarkus sobe uma vez para todos os testes e só reinicia quando a configuração muda (por exemplo, perfis de teste diferentes). O estado dos beans é compartilhado entre testes.
- A porta de teste padrão é `8081`, a mesma que o exemplo usa para a aplicação (seção 3.6).
- Para testar o recurso sem subir o Quarkus, a seção 7.6 mostra um teste JUnit puro. Para trocar um bean dentro do `@QuarkusTest`, a documentação oferece `@Alternative` em classes de teste, `QuarkusMock` e `@InjectMock`.

> Erro comum (Quarkus): teste que depende da ordem. Como a aplicação e o repositório são os mesmos para todos os testes, um teste que conta tarefas passa ou falha conforme os `POST` de outros testes rodaram antes. No exemplo acima, `Location` termina em `/tarefas/3` só porque nenhum outro teste criou tarefa antes. Teste propriedades que não dependem do histórico (o status, o formato, a tarefa que o próprio teste criou) ou limpe o estado antes de cada teste.

📖 Ref. Quarkus — Testing your application: https://quarkus.io/guides/getting-started-testing

10.5 Tabela final

Tema	Ktor	Quarkus	Capítulo
Natureza	biblioteca: você monta	framework: convenções e build	1
Rotas	DSL de código (`routing` , `route` , `get`)	anotações (`@Path` , `@GET`)	2
Parâmetros	`call.parameters` , conversão manual	`@RestPath` , `@RestQuery` , conversão automática	2
Barra final, método não suportado	`404` , `404`	`200` , `405`	2
Configuração	código ou `application.conf`	`application.properties` e perfis	3
JSON	kotlinx.serialization, código gerado	Jackson, reflexão	5
JSON inesperado	rígido: `400`	tolerante: ignora chaves, aceita `null`	5
Corpo	`call.receive<T>()`	parâmetro do método	6
Status e `Location`	`respond(status, corpo)` , `response.header`	`Response.created(uri)` , `RestResponse<T>`	6
Erros centralizados	`StatusPages`	`@ServerExceptionMapper`	6
Validação	`RequestValidation`	Hibernate Validator	6
Injeção	Koin, grafo em código, erros em execução	CDI, anotações, erros no build	8
Espera de E/S	corrotinas; bloqueio isolado em `Dispatchers.IO`	threads de trabalho; `Uni` no event loop; threads virtuais	9
Testes	`testApplication` , em memória, estado novo por teste	`@QuarkusTest` + RestAssured, aplicação compartilhada	10

10.6 Como escolher o stack do grupo

A mesma requisição produz a mesma resposta nos dois. A escolha é sobre o grupo e o produto:

- Linguagem que o grupo domina ou quer dominar. Kotlin tem curva extra (capítulo 4); Java com anotações tem curva de convenções (capítulos 2, 3 e 8).
- Preferência por ver tudo no código (Ktor) ou por convenções que escrevem a montagem (Quarkus).
- Ecossistema necessário. Extensões prontas do Quarkus (OpenAPI, validação, persistência, segurança) × plugins do Ktor mais bibliotecas Kotlin.
- Onde o erro de montagem deve aparecer: no build (CDI) ou em execução e testes (Koin).

Qualquer escolha atende a rubrica. O que pesa na avaliação é o contrato HTTP, a arquitetura e os testes, que este guia tratou nos dois.

10.7 Leitura recomendada

- Ktor — Creating a new project: https://ktor.io/docs/server-create-a-new-project.html
- Quarkus — Getting started: https://quarkus.io/guides/getting-started
- Kotlin — Coroutines overview: https://kotlinlang.org/docs/coroutines-overview.html
- RFC 9110 (semântica HTTP) e RFC 9457 (problem details), já citadas nos capítulos 1 e 6.
- MUSI: `api-ktor/` e `api-quarkus/`, as mesmas peças num projeto maior.

Próxima aula (21/09): o repositório em memória vira banco (Exposed × Hibernate/Panache, com Flyway), os testes passam a usar Testcontainers, e a API ganha documentação OpenAPI.

---

11. Exercícios e dúvidas frequentes

11.1 Perguntas de fixação

Tente responder antes de abrir a resposta indicada.

1. Por que repetir um `POST /tarefas` é diferente de repetir um `PUT /tarefas/3`? (1.3)
2. Um cliente envia JSON sem `Content-Type`. Que status os dois exemplos devolvem, e por quê? (1.5, 5.3)
3. O que `call` é, dentro de `get("/tarefas") { ... }`, e por que ele aparece sem declaração? (4.6)
4. Por que `call.receive<NovaTarefa>()` não precisa de `NovaTarefa::class`, enquanto o Jackson pede `NovaTarefa.class`? (4.10)
5. `json(Json { prettyPrint = true })` fez o campo `feita` sumir. Explique a causa e a correção. (5.4)

6. Um `POST` com `{}` gera tarefa com título `null` no Quarkus e `400` no Ktor. Qual característica de cada stack explica a diferença? (5.6)
7. Em que situação `400` é mais adequado que `422`? (6.7)
8. A rota depende do repositório em execução. Por que se diz que a dependência de código aponta para o domínio? (7.2)
9. Qual a diferença de momento de erro entre `by inject()` e `get()` no Koin, e entre Koin e CDI? (8.3, 8.6)
10. Um método Quarkus que devolve `List<Tarefa>` roda no event loop? (9.4)
11. Por que `withContext(Dispatchers.IO)` não aumentou a capacidade de 64 esperas simultâneas no teste? (9.3)
12. Por que o repositório em memória perdeu tarefas sob carga, se todas as respostas foram `201`? (9.5)

11.2 Exercícios práticos

Faça no stack do grupo. Cada exercício tem um critério de pronto verificável com `curl -i` ou com um teste.

1. Buscar por id. `GET /tarefas/{id}` devolve `200` com a tarefa, `404` em `application/problem+json` quando não existe, `400` quando o id não é número. Pronto quando os três casos tiverem teste automatizado.

2. `Location` no `POST`. Pronto quando `curl -i -X POST ...` mostrar `201` e `Location` apontando para uma URI que, chamada com `GET`, devolve a tarefa criada.

3. Remover. `DELETE /tarefas/{id}` devolve `204`; repetir devolve `404`. Explique no README por que isso não fere a idempotência.

4. Substituir com `PUT`. `PUT /tarefas/{id}` com corpo `{"titulo":"...", "feita":true}` substitui a tarefa. Decida (e documente) se `PUT` num id inexistente cria ou devolve `404`. Pronto quando duas chamadas idênticas deixarem o mesmo estado.

5. Filtro e paginação. `GET /tarefas?feita=true&pagina=0&tamanho=10`. Valores inválidos (`tamanho=-1`, `feita=talvez`) devolvem `400` em problem+json, não são ignorados. Pronto com teste para um valor válido e um inválido de cada parâmetro.

6. Validação. Título em branco ou com mais de 200 caracteres devolve `422` em problem+json, com `detail` dizendo qual regra falhou.

7. Tratador genérico sem regressão. Instale o tratador de `Throwable` (Ktor) ou um mapeador de `Exception` (Quarkus) e confirme, com teste, que `Content-Type: text/plain` continua devolvendo `415`, e não `500`.

8. Repositório falso. Escreva um teste da rota (Ktor) ou do recurso (Quarkus) usando um repositório falso, sem o repositório em memória. Pronto quando o teste rodar sem subir banco nem Koin/CDI reais.

9. Repositório seguro para concorrência. Reproduza o teste da seção 9.5 (2.000 `POST`, 200 simultâneos), confirme a perda de tarefas e corrija. Pronto quando a contagem bater em três execuções seguidas.

10. Experimento de bloqueio. Crie os endpoints de espera do capítulo 9 no seu stack, meça 300 requisições simultâneas e registre os tempos no README, com a sua interpretação.

11. Injeção quebrada de propósito. Ktor: remova a definição do repositório e escreva um teste com `verify()` que detecte o problema (use `singleOf` com construtor). Quarkus: crie uma segunda implementação do repositório, leia a mensagem do build e resolva com `@Named`.

12. Escopo por requisição. Crie um objeto com escopo de requisição que guarda o instante de início e acrescente um cabeçalho `X-Tempo-Ms` à resposta. Koin: `requestScope`. Quarkus: `@RequestScoped`.

11.3 Dúvidas frequentes

- *Meu `POST` devolve `415 Unsupported Media Type`.* Falta `-H 'Content-Type: application/json'`. O `curl -d` envia `application/x-www-form-urlencoded` (1.5).
- *O campo `feita` sumiu do JSON no Ktor.* A instância de `Json` foi criada com `Json { }` e perdeu o `encodeDefaults` do `DefaultJson`. Use `Json(DefaultJson) { ... }` (5.4).
- *`GET /tarefas/` dá `404` no Ktor e funciona no Quarkus.* O Ktor distingue barra final; o Quarkus não. Instale `IgnoreTrailingSlash` ou padronize os caminhos sem barra (2.6).
- *O Ktor não lê meu `application.conf`.* O servidor sobe com `embeddedServer`, que não lê o arquivo. Use `EngineMain` ou variáveis de ambiente (3.2).
- *`mvn test` falha com `Port already bound: 8081`.* O dev mode está usando a 8081, que também é a porta padrão dos testes. Defina `quarkus.http.test-port` (3.6).
- *O build do Quarkus falha com `UnsatisfiedResolutionException`.* A implementação não tem anotação de escopo, ou está fora do que o Quarkus indexa (8.4).
- *O build falha com `AmbiguousResolutionException`.* Há duas implementações da interface. Escolha uma com `@Named`, `@Alternative` ou `@DefaultBean` (8.4).
- *`NoDefinitionFoundException` na primeira requisição (Koin).* Falta a definição, ou ela foi registrada só pelo tipo concreto (`singleOf` sem `bind`) (8.3, 8.7).
- *`call` aparece "do nada" no handler.* O handler é uma lambda com receptor `RoutingContext`, e `call` é propriedade dele (4.6).
- *Preciso de `suspend` nas funções do repositório?* Só se a implementação suspender de verdade. Com JDBC, a chamada é bloqueante; isole em `Dispatchers.IO` (9.2, 9.3).
- *`@ApplicationScoped` ou `@Singleton`?* `@ApplicationScoped`, salvo motivo específico. Ele é preguiçoso, permite mocks e injeção de escopos menores (8.4).

- *`400` ou `422` para validação?* `400` quando a requisição não pôde ser interpretada; `422` quando foi interpretada e viola uma regra. O Quarkus usa `400` por padrão; os exemplos da Sprint 0 usam `422`. Escolha e seja consistente (6.7).
- *`record` ou classe com getters para os tipos Java?* `record` para dados imutáveis de entrada e saída. O Jackson lida com records sem configuração (4.8, 5.5).
- *Por que meu teste Quarkus passa sozinho e falha junto com os outros?* A aplicação sobe uma vez para todos os testes; o estado dos beans é compartilhado (10.4).
- *O repositório em memória "perde" tarefas.* Acesso concorrente sem sincronização (9.5).

Referências

HTTP

- RFC 9110 — HTTP Semantics: https://www.rfc-editor.org/rfc/rfc9110.html
- MDN — Visão geral do HTTP (PT): https://developer.mozilla.org/pt-BR/docs/Web/HTTP/Guides/Overview
- MDN — Mensagens HTTP (PT): https://developer.mozilla.org/pt-BR/docs/Web/HTTP/Guides/Messages
- MDN — Métodos (PT): https://developer.mozilla.org/pt-BR/docs/Web/HTTP/Reference/Methods
- MDN — Códigos de status (PT): https://developer.mozilla.org/pt-BR/docs/Web/HTTP/Reference/Status
- MDN — Negociação de conteúdo (PT): https://developer.mozilla.org/pt-BR/docs/Web/HTTP/Guides/Content_negotiation
- MDN — Location (PT): https://developer.mozilla.org/pt-BR/docs/Web/HTTP/Reference/Headers/Location
- MDN — 204 No Content (PT): https://developer.mozilla.org/pt-BR/docs/Web/HTTP/Reference/Status/204
- RFC 9457 — Problem Details for HTTP APIs: https://www.rfc-editor.org/rfc/rfc9457.html
- curl — manual: https://curl.se/docs/manpage.html

Kotlin

- Basic syntax: https://kotlinlang.org/docs/basic-syntax.html
- Null safety: https://kotlinlang.org/docs/null-safety.html
- Nulidade, de Java para Kotlin: https://kotlinlang.org/docs/java-to-kotlin-nullability-guide.html
- Coleções, de Java para Kotlin: https://kotlinlang.org/docs/java-to-kotlin-collections-guide.html
- Functions (argumentos nomeados e padrão): https://kotlinlang.org/docs/functions.html
- Chamando Kotlin a partir de Java: https://kotlinlang.org/docs/java-to-kotlin-interop.html
- Lambdas, trailing lambdas e lambda com receptor: https://kotlinlang.org/docs/lambdas.html
- Functional (SAM) interfaces: https://kotlinlang.org/docs/fun-interfaces.html
- Extensions: https://kotlinlang.org/docs/extensions.html

- Type-safe builders (DSL): https://kotlinlang.org/docs/type-safe-builders.html
- Scope functions: https://kotlinlang.org/docs/scope-functions.html
- Data classes: https://kotlinlang.org/docs/data-classes.html
- Delegated properties: https://kotlinlang.org/docs/delegated-properties.html
- Generics / type erasure: https://kotlinlang.org/docs/generics.html#type-erasure
- Inline / reified: https://kotlinlang.org/docs/inline-functions.html#reified-type-parameters
- Comparison to Java: https://kotlinlang.org/docs/comparison-to-java.html
- Serialization: https://kotlinlang.org/docs/serialization.html
- kotlinx.serialization — Basic serialization: https://github.com/Kotlin/kotlinx.serialization/blob/master/docs/basic-serialization.md
- kotlinx.serialization — Json configuration: https://github.com/Kotlin/kotlinx.serialization/blob/master/docs/json.md
- Coroutines (visão geral): https://kotlinlang.org/docs/coroutines-overview.html
- Coroutines basics: https://kotlinlang.org/docs/coroutines-basics.html
- Coroutine context and dispatchers: https://kotlinlang.org/docs/coroutine-context-and-dispatchers.html
- kotlinx.coroutines — `Dispatchers.IO`: https://kotlinlang.org/api/kotlinx.coroutines/kotlinx-coroutines-core/kotlinx.coroutines/-dispatchers/-i-o.html

Java

- Record classes (Java 25): https://docs.oracle.com/en/java/javase/25/language/records.html
- JEP 444 — Virtual Threads: https://openjdk.org/jeps/444
- Jakarta Servlet: https://jakarta.ee/specifications/servlet/

Ktor

- Criar um projeto: https://ktor.io/docs/server-create-a-new-project.html
- Engines: https://ktor.io/docs/server-engines.html
- Configuração em código: https://ktor.io/docs/server-configuration-code.html
- Configuração em arquivo: https://ktor.io/docs/server-configuration-file.html
- Módulos: https://ktor.io/docs/server-modules.html
- Development mode: https://ktor.io/docs/server-development-mode.html
- Rotas: https://ktor.io/docs/server-routing.html
- Requisições / respostas: https://ktor.io/docs/server-requests.html · https://ktor.io/docs/server-responses.html
- Serialização / content negotiation: https://ktor.io/docs/server-serialization.html
- Request validation: https://ktor.io/docs/server-request-validation.html
- Status pages: https://ktor.io/docs/server-status-pages.html
- Testing: https://ktor.io/docs/server-testing.html
- Dependency injection (plugin próprio): https://ktor.io/docs/server-dependency-injection.html
- Koin com Ktor (quickstart): https://insert-koin.io/docs/quickstart/ktor

- Koin — Definitions: https://insert-koin.io/docs/reference/koin-core/definitions
- Koin — Koin for Ktor: https://insert-koin.io/docs/reference/koin-ktor/ktor
- Koin — Request scopes: https://insert-koin.io/docs/reference/koin-ktor/ktor-scopes
- Koin — Testing Ktor with Koin: https://insert-koin.io/docs/reference/koin-ktor/ktor-testing
- Koin — Verify: https://insert-koin.io/docs/reference/koin-test/verify

Quarkus

- Getting started: https://quarkus.io/guides/getting-started
- Quarkus REST: https://quarkus.io/guides/rest
- JSON REST: https://quarkus.io/guides/rest-json
- Validation: https://quarkus.io/guides/validation
- Configuração (fontes e perfis): https://quarkus.io/guides/config-reference
- HTTP reference: https://quarkus.io/guides/http-reference
- Continuous testing: https://quarkus.io/guides/continuous-testing
- CDI: https://quarkus.io/guides/cdi
- CDI reference: https://quarkus.io/guides/cdi-reference
- Testing your application: https://quarkus.io/guides/getting-started-testing
- Virtual thread support: https://quarkus.io/guides/virtual-threads
- Arquitetura reativa: https://quarkus.io/guides/quarkus-reactive-architecture

Arquitetura

- Fowler — Inversion of Control (EN): https://martinfowler.com/bliki/InversionOfControl.html
- Fowler — Inversion of Control Containers and the Dependency Injection pattern (EN): https://martinfowler.com/articles/injection.html
- Fowler — Test Double (EN): https://martinfowler.com/bliki/TestDouble.html
- Robert C. Martin — The Clean Architecture (EN): https://blog.cleancoder.com/uncle-bob/2012/08/13/the-clean-architecture.html
- Alistair Cockburn — Hexagonal Architecture (EN): https://alistair.cockburn.us/hexagonal-architecture/
- Spring — Dependency Injection (EN): https://docs.spring.io/spring-framework/reference/core/beans/dependencies/factory-collaborators.html
- ArchUnit: https://www.archunit.org/ · Konsist: https://docs.konsist.lemonappdev.com/
- IntelliJ IDEA — HTTP Client: https://www.jetbrains.com/help/idea/http-client-in-product-code-editor.html
- Fowler — Repository (EN): https://martinfowler.com/eaaCatalog/repository.html
- Microsoft Learn — Padrão de repositório (PT): https://learn.microsoft.com/pt-br/dotnet/architecture/microservices/microservice-ddd-cqrs-patterns/infrastructure-persistence-layer-design
- Jakarta REST (`Response`): https://jakarta.ee/specifications/restful-ws/ · Jakarta CDI: https://jakarta.ee/specifications/cdi/